VERINF: ZERO-KNOWLEDGE PROOFS OF INFERENCE FOR FRONTIER-SCALE MODELS ON A SINGLE MACHINE

James Petrie *

ABSTRACT

We present a hash-based zero-knowledge argument whose streaming implementation lets us prove a 400-billion-parameter model on a consumer device. The argument bounds the unexplained information in an observed output stream, certifying it against a committed integer model without simulating floating point. The construction uniquely pins every witness upstream of the logits and makes each downstream freedom provably inflate the report, so the certified bound cannot be deflated by the prover. We demonstrate a proof of a 1000-token forward pass of Llama-4-Maverick, a mixture-of-experts model, generated in 14.3 hours on a single NVIDIA DGX Spark and accepted by an independently implemented verifier. The prover streams a 7.2-terabyte witness at an 83.9 GB working set; at 400 billion parameters, the proof is roughly thirty times larger than prior published zero-knowledge inference. A cost model calibrated against measured hardware performance projects that a million-token context with dense attention in every layer could be proven in days on an NVL72-class cluster. Together these make the argument a candidate primitive for high-stakes agreements about AI between well-resourced parties who do not trust each other's hardware, a setting that also makes an interactive protocol natural and large proofs and slow verification acceptable.

1. Introduction

VerInf verifies how much of the information in a token stream is explained by inference of a committed large language model, in zero knowledge. Its one public quantity is a bound on the unexplained information in the output, the bits that a permitted computation on the measured inputs does not account for. The committed witness at frontier scale exceeds any single machine's memory, so our implementation streams it operation by operation. This is what makes massive proofs feasible on small hardware: VerInf produced a proof of a 1000-token forward pass of Llama-4-Maverick, a 400-billion-parameter mixture of experts with all 128 experts committed per layer, in 14.3 hours on a single consumer NVIDIA DGX Spark, streaming a 7.2-terabyte witness at a peak working set of 83.9 GB. Prior published zero-knowledge inference work reaches models of about 13 billion parameters (Sun, Li, and Zhang 2024), roughly thirty times smaller.

Deployed inference typically runs in floating point and is typically nondeterministic across executions. Prior inference proofs require the computation to match an integer circuit exactly, so they attest to a computation contrived to fit the proof; how easily a given serving stack could be made bit-exact is difficult to judge from outside. VerInf requires no changes to the deployment. Rather than reproduce its output, the proof certifies how well a committed integer version of the model predicts it, and the cost of every approximation appears in the reported bound rather than in a failed proof. A deployment that does run deterministically needs no new machinery and simply certifies a tighter bound.

The target use case is sampling packets under high-stakes AI agreements (Shavit 2023; Wasil et al. 2024; Scher and Thiergart 2024; Baker et al. 2025). In the companion framework [cite], a compute operator (the prover) demonstrates to an external party (the verifier) that it runs only permitted workloads, where neither party trusts the other's hardware; the framework gives three confidentiality-preserving ways to perform the recomputation, and this paper builds the zero-knowledge one. Three properties of the setting shape the design. Only randomly sampled outputs are proven, so proving cost amortizes over the workload it deters rather than gating it. Both parties have substantial resources, so slow proving, slow verification, and large proofs are all acceptable, while a small trusted base matters greatly; the demonstrated proof is 93.6 GB and takes hours to check. And the parties are in live contact, so the proof can be

*james.petrie@protonmail.com

interactive: the prover commits before each challenge is drawn and cannot grind over challenges offline, which lets a modest per-challenge soundness level carry real deterrent weight.

The bound itself is a sum of per-token surprisals under a predictor of the deployment's outputs that the prover supplies. By Gibbs' inequality the sum is a valid upper bound on the unexplained information for any predictor, so the design of the predictor can be left to the prover, which is best placed to make it accurate; a poor predictor inflates only the prover's own reported number. Beyond the bound and the public claim list, the proof reveals nothing: not the weights, the activations, or the input and output tokens. The bound places an unusual demand on the proof system: the prover must have no freedom it can use to deflate it. Witness slack is a liability for any inference proof, since slack in an intermediate value propagates through the remaining layers in ways that are hard to analyze; for a bound on unexplained information it is directly exploitable, since the prover selects among valid witnesses in the direction that inflates its predicted probabilities. VerInf therefore meets a two-tier requirement: upstream of the logits every claim admits a unique witness, and downstream of the logits every prover freedom provably inflates the reported bound (§7.2).

VerInf uses Ligero (Ames et al. 2017) as its argument system because it is transparent, needing no trusted setup, plausibly post-quantum, with commitments that are hash-based only, and among the simplest constructions available. The trade Ligero makes is proof size and verifier work that grow as the square root of the witness rather than polylogarithmically, which the setting tolerates: verification is occasional and offline, and the verifier is well resourced. A cost model fitted to measured hardware primitives and validated against the demonstrated runs projects that a one-million-token context could be proven in days on an NVL72-class cluster; the projection assumes dense attention in every layer, and windowed or sparse attention reduces the dominant quadratic term in proportion to the layers and span it covers (§8). The verifier recompiles every constraint from the public claim list and checks the proof only against its own derivation, never against constraints supplied with the proof, so its trusted base is short and auditable.

A model is written as ordinary tensor code against a tape, in the style of PyTorch; each operation records a public claim, and each claim type carries the machinery to prove it, some of it using an additional challenge round. Matrix multiplications are proven by Freivalds' check: the prover commits the input and output matrices, the verifier supplies random projection vectors, and the prover shows the projection of the committed output equals the product of the projections of the committed inputs, reducing each matmul to a single short dot product. Nonlinearities such as softmax are proven against public lookup tables with LogUp (Haböck 2022), whose cost amortizes across the many operations sharing a table. Softmax is where the unique-witness requirement trades against efficiency: its per-row shift is not an integer, and constraining it only to a tolerance band would leave the prover a choice among valid witnesses. VerInf instead pins the shift to a unique integer using the monotonicity of the row sum, at the cost of a second table and the bracket constraints (§3.5). The bound is meaningful only for the transcript the run actually produced, so the committed token streams are bound to digests recorded independently at generation time (§7.2); the circuits for this binding are implemented, and demonstrating it end to end against recorded digests is future work.

The demonstrated system has two main limitations. The committed integer model leaves 0.880 bits per token unexplained, explaining about 95% of the information a token from the 202,048-token vocabulary can carry; future work can tighten this by modeling the deployment's floating-point computation more closely. And the public claim list reveals the model architecture, though not the weights; a second proof stage, showing that the verifier's own architecture checks ran and accepted, could hide it (§10).

Our contributions are:

- A proof design that leaves the prover no freedom to deflate the reported bound, with a unique witness pinned at every step of the forward pass.
- A complete implementation on a hash-based proof system (Ligero), with no trusted setup: a streaming prover whose peak memory tracks one operation rather than the witness, and a verifier that recompiles every constraint from the public claim list.
- A demonstration at frontier scale: a proof of a 1000-token forward pass of a 400-billion-parameter mixture of experts, generated in 14.3 hours on a single consumer machine by streaming the 7.2-terabyte witness at a working set of 83.9 GB, and accepted by an independently implemented verifier.
- A cost model fitted to measured hardware primitives, projecting that a million-token context with dense attention in every layer could be proven in days on an NVL72-class cluster (§8).

The paper is organized as follows. Section 2 compares with prior work. Section 3 gives the proof system claim by claim, Section 4 the unexplained-information bound and its computation inside the proof, and Section 5 the claim format. Section 6 describes the implementation, Section 7 the soundness analysis, together with the trust model and the anchoring of the committed transcript, Section 8 the cost model and scaling, Section 9 the results, and Section 10 the limitations and future work.

2. Related work

Zero-knowledge proofs of LLM inference. zkLLM (Sun, Li, and Zhang 2024) is the closest prior work: a sumcheck-based argument with a tailored proof of attention, proving LLaMA-2-13B at a 2,048-token sequence in about 13 minutes on an A100. VerInf differs on three axes. On scale, its demonstrated run is roughly thirty times larger; the reported zkLLM system demonstrates the prover, whereas the demonstrated VerInf runs are end to end, with an independently implemented verifier accepting the proof and the zero-knowledge masking active. On the statement proven, zkLLM, like the other systems below, requires the computation to match its integer circuit exactly, so it attests to a computation contrived to fit the proof; VerInf proves that the committed output is well explained by the committed integer model and bounds what goes unexplained, which is what lets it target a deployment as it runs (§1). And on commitments, zkLLM instantiates its polynomial commitment with Hyrax (Wahby et al. 2018), which rests on the discrete-logarithm assumption; VerInf's commitments are hash-based only, with no trusted setup and plausibly post-quantum.

zkGPT (Qu et al. 2025) proves GPT-2 inference in under 25 seconds; zkPyTorch (Xie et al. 2025) compiles quantized models to a GKR pipeline, reporting Llama-3-8B at 150 seconds per token on one CPU thread; ZKTorch (Chen, Tang, and Kang 2025) compiles inference to an accumulation scheme at the several-billion-parameter scale. The line descends from pre-LLM zkML systems such as zkCNN (Liu, Xie, and Zhang 2021) and ZKML (Chen et al. 2024); Peng et al. (2025) survey the area.

The exact-circuit systems also leave the prover freedom in the witness. zkLLM constrains the softmax normalization only to a tolerance band on the row sum, leaves the rounding rule for table entries unspecified, and lets the prover choose some setup parameters; each admits multiple satisfying witnesses, so the proven relation is weaker than exact execution of the committed circuit, and how much the accumulated slack admits has not been analyzed. For the statement VerInf proves the question cannot be left open, since any such freedom is a direct lever on the reported bound (§7.2). The construction therefore replaces each with a public constant or a unique-witness gadget: the tables, scales, and noise parameters are constants of the claim list, every table entry follows a deterministic rounding rule the verifier reproduces, and the softmax shift is pinned by a two-table bracket (§3.5, Appendix B).

Verified inference with a trusted verifier. A separate line verifies inference in the unilateral setting, where the verifier is trusted and sees plaintext. Rinberg et al. (2025) and Karvonen et al. (2025), companion works, verify recomputation against hardware nondeterminism to detect weight exfiltration and inference-specification violations; VerInf shares their per-token surprisal measure under a logit-noise model. TOPLOC (Ong et al. 2025) attests with locality-sensitive hashes of activations, and Verde (Arun et al. 2025) instead makes inference bitwise reproducible so that arbitration can settle disputes. VerInf targets the bilateral setting, where the verifier trusts neither the prover's hardware nor its software and must not see the tokens: the recomputation is replaced by a zero-knowledge proof, and the verifier learns only the bound.

3. Proof system

VerInf is built on the Ligero zero-knowledge argument system (Ames et al. 2017), which supports commitments, linear tests, and Hadamard tests, all in zero knowledge, with security resting only on the collision resistance of a hash function. This section describes these tools (§3.1); how matrix multiplication (§3.2), mixture-of-experts routing (§3.3), table lookups (§3.4), and softmax (§3.5) are proven with them; and rescaling and overflow handling (§3.6). Every claim's full listing and soundness lemma is in Appendix B.

3.1 Ligero overview

Ligero provides three tools, and everything in the paper is built from them:

- **Zero-knowledge commitment.** A short commitment binds a block of field values while revealing nothing about them. Commitments can be issued across turns, and a proof can span several blocks, including a persistent block reused across proofs (the model weights, §6.4).
- **Linear test.** Demonstrates that $M\boldsymbol{x} = v$ holds for public M, v and hidden $\boldsymbol{x}$.
- **Hadamard test.** Demonstrates that committed vectors satisfy $\boldsymbol{x} \circ \boldsymbol{y} = \boldsymbol{z}$, the elementwise (Hadamard) product, with $\boldsymbol{x}$, $\boldsymbol{y}$, and $\boldsymbol{z}$ all hidden. Every quadratic constraint in the paper is of this form.

The base Ligero protocol runs in three rounds: (1) the prover commits; (2) the verifier sends random challenges, and the prover folds all constraints into a few short test results; (3) the verifier picks random locations of the commitment to audit, and the prover opens them, letting the verifier check that the test results are consistent with the committed values.

Every committed value is a 64-bit fixed-point integer at the working scale $\mathrm{S} = 2^{12}$, in the Goldilocks field $|F| = 2^{64} - 2^{32} + 1$; the lookups and brackets of the checks below need integer keys and integer comparisons, and fixed point provides them. Products of fixed-point values arrive at a higher scale and are returned to the working scale by a rescaling operation (§3.6).

3.2 Matrix multiplication

Consider a committed product $\boldsymbol{C} = \boldsymbol{A}\boldsymbol{B}$ of shapes (m, k) and (k, n). Checking it entrywise would need mnk Hadamard constraints. VerInf instead checks the random projection

$$\lambda^\top \boldsymbol{C}\, \rho \;=\; \lambda^\top (\boldsymbol{A}\boldsymbol{B})\, \rho,$$

with $\rho \in F^n$ and $\lambda \in F^m$ supplied by the verifier after $\boldsymbol{A}$, $\boldsymbol{B}$, and $\boldsymbol{C}$ are committed; a wrong $\boldsymbol{C}$ survives with probability at most $2/|F|$. To evaluate the right side, the prover commits three length-k vectors, $\boldsymbol{y} = \boldsymbol{B}\rho$, $\boldsymbol{u} = \lambda^\top \boldsymbol{A}$, and $\boldsymbol{p} = \boldsymbol{u} \circ \boldsymbol{y}$, enforced by two linear tests and one Hadamard test. The identity is then the single linear constraint $\sum_{a,b} \lambda[a]\, \rho[b]\, \boldsymbol{C}[a, b] = \sum_j \boldsymbol{p}[j]$. In total, $3k$ committed values, $2k + 1$ linear constraints, and k Hadamard constraints replace the full check's mnk committed products, mnk Hadamard constraints, and mn linear sums. Details, including the rescale of the output (§3.6), are in Appendix B.2.

3.3 Mixture-of-experts routing

In a mixture-of-experts layer each token routes to one of E experts. The routing decision is a committed one-hot vector $\boldsymbol{m} \in \{0, 1\}^E$ per token: the Hadamard test $\boldsymbol{m} \circ \boldsymbol{m} = \boldsymbol{m}$ forces every entry to 0 or 1, the linear test $\sum_e \boldsymbol{m}[e] = 1$ forces exactly one 1, and a range-checked gap constraint (§3.4) forces the 1 to the argmax of the router logits, tiebroken by expert index. The layer output $\boldsymbol{y} = \sum_e \boldsymbol{m}[e]\, \boldsymbol{x}_e$ over the E committed expert outputs $\boldsymbol{x}_e$ is checked by projection, as with matmul: for a verifier-supplied ρ, the prover commits each projection $\boldsymbol{s}_e = \boldsymbol{x}_e \cdot \rho$ (linear tests), the masked values $\boldsymbol{m}[e]\, \boldsymbol{s}_e$ (Hadamard tests), and one linear constraint equates their sum with $\boldsymbol{y} \cdot \rho$. All E expert outputs are committed even though one fires; committing only the active expert would reveal the route (B.6).

3.4 Lookup tables

A nonlinearity $\boldsymbol{y}_i = f(\boldsymbol{x}_i)$, such as the exponential, is proven by lookup against the public table of all input-output pairs $(t_j, f(t_j))$. LogUp (Haböck 2022) reduces a batch of queries to the single identity

$$\sum_i \frac{1}{\beta + \boldsymbol{x}_i + \alpha\, \boldsymbol{y}_i} \;=\; \sum_j \frac{\boldsymbol{m}_j}{\beta + t_j + \alpha\, f(t_j)},$$

over random challenges α, β, with the multiplicities $\boldsymbol{m}_j$ (how often each entry is queried) committed before α, β are drawn and the per-query inverses committed after. The challenge folds key and value together, so one lookup certifies both that $\boldsymbol{x}_i$ is in the table's range and that $\boldsymbol{y}_i = f(\boldsymbol{x}_i)$. Tables are computed by a deterministic rounding rule the verifier reproduces.

3.5 Softmax

Per row of scores $\boldsymbol{x}$, softmax must produce

$$\boldsymbol{y}[i] \;=\; \frac{e^{\boldsymbol{x}[i]}}{\sum_j e^{\boldsymbol{x}[j]}}.$$

Following zkLLM (Sun, Li, and Zhang 2024), the prover proposes a per-row shift $\boldsymbol{c}$ instead of verifying the division, and each output is computed by table lookup (§3.4) as $\boldsymbol{y}[i] = e^{\boldsymbol{x}[i] - \boldsymbol{c}}$. Softmax is shift-invariant, so this is correct exactly when $\boldsymbol{c}$ equals the row's log-sum-exp, which makes the outputs sum to the normalization target (one, up to quantization). The true log-sum-exp is not an integer, so an exact check on it is impossible. zkLLM accepts any $\boldsymbol{c}$ whose outputs sum to within a tolerance of the target: the tolerance absorbs rounding error, but it also admits several neighboring integer shifts, and the prover can pick whichever nudges the outputs in its favor, a direct lever on the bound (§7.2). VerInf instead requires the sum at $\boldsymbol{c}$ to be at most the target and the sum at $\boldsymbol{c} - 1$ to exceed it. Raising the shift shrinks every exponential, so the row sum is monotonically decreasing in $\boldsymbol{c}$: $\boldsymbol{c}$ must be the first shift at which the sum

crosses the target, a monotonic sequence crosses a threshold only once, and the satisfying integer is therefore unique. The sum at $\boldsymbol{c} - 1$ is evaluated against a second copy of the exponential table shifted by one index, so both sums are exact table sums. The full claim, with the saturation of large scores and the causal mask compiled in as public structure, is B.3.

3.6 Rescaling and overflow

A product of two values at the working scale arrives at scale S^2. Each multiplicative claim returns its output to the working scale by a *rescale*: the committed raw product $\boldsymbol{x}_{\text{full}}$ is split into a kept word $\boldsymbol{x}$ and dropped low bits $\boldsymbol{x}_{\text{low}}$ by one linear constraint and two range memberships,

$$\boldsymbol{x}_{\text{full}} \;=\; 2^{12}\,\boldsymbol{x} + \boldsymbol{x}_{\text{low}}, \qquad \boldsymbol{x}_{\text{low}} \in [0,\ 2^{12}), \qquad \boldsymbol{x} + 2^{25} \in [0,\ 2^{26}),$$

each membership one lookup (§3.4) against the table of every integer in its range, the second checked on a shifted copy so that the signed output lies in $[-2^{25}, 2^{25})$. The linear constraint fixes the weighted sum, the range checks confine each word to its window, and the windows tile, so exactly one assignment satisfies all three. The products themselves are computed exactly, since every accumulator stays below 2^{53}; the accuracy cost of the representation is the dropped bits, priced by the bound (§4).

The central numeric constraint is overflow: every committed value and every intermediate product must stay below the modulus. This is what fixes the scale choices, and what forces a rescale into a claim when an input arrives at a higher scale than the claim can absorb.

If a value nevertheless overflows, one of two things happens. Where it feeds a decomposition like the one above, the wrapped field element has no valid split into range-checked words, and the proof rejects. Elsewhere the constraints are field identities, so the proof accepts and attests the wrapped computation. This costs accuracy, not soundness: the wrapped computation is still deterministic, with a unique witness. In practice its logits predict the deployment's tokens poorly, and the divergence surfaces as a large bound. Overflow needs careful management in two places: in arguments that rely on monotonicity, such as the softmax bracket (§3.5, B.3), and in the unexplained-information claims, where wraparound must not let the prover deflate the bound (§4, B.7).

4. Bounding unexplained information

VerInf attests to a single public quantity: a bound on the unexplained information in a committed output, given a committed model and committed inputs. The fewer bits of the output stream left unexplained, the more closely the stream matches what the committed model, run on the committed inputs, would produce.

The declared computation D is the workload the prover claims to be running; in this paper it is the committed integer model of §3. Running D on the measured inputs $\boldsymbol{x}$ produces $D(\boldsymbol{x})$, which includes intermediate values such as the logits. The companion paper [cite] defines the true unexplained information U as the conditional entropy of the output given $D(\boldsymbol{x})$, and bounds it, in expectation over the hardware's randomness, by a sum of per-token surprisals under any predictor:

$$U \;\le\; \mathbb{E}\left[-\textstyle\sum_t \log_2 Q\left(\boldsymbol{o}_t \mid D(\boldsymbol{x}),\ \boldsymbol{o}_{<t}\right)\right].$$

Here Q is any fixed conditional distribution predicting each output token from $D(\boldsymbol{x})$ and the earlier tokens; $\boldsymbol{o}$ denotes the output stream, and the measured output is the realization the proof scores. The proof reports $\hat{U}$: the same surprisal sum, evaluated on the measured output inside the proof with every rounding pushed upward, and published through a fixed public conversion to bits (B.7), so that

$$-\textstyle\sum_t \log_2 Q\left(\boldsymbol{o}_t \mid D(\boldsymbol{x}),\ \boldsymbol{o}_{<t}\right) \;\le\; \hat{U}.$$

$\hat{U}$ is the proof's only public output, an upper bound on the true unexplained information; the weights, inputs, and output tokens stay hidden. The derivation, the extension to non-zero-temperature sampling, and the security analysis are in the companion paper. Any Q gives a valid bound, and a poor one inflates only the prover's own reported number, so the predictor can be left to the prover; it is committed before any challenge is drawn, and the proof certifies that the reported value is at least the surprisal under it (B.7). Our prototype models hardware nondeterminism as Gaussian noise on the logits, $Q(\boldsymbol{o}_t \mid \boldsymbol{\ell}_t) \propto \exp(-(v^* - \boldsymbol{\ell}_t[\boldsymbol{o}_t])^2/\sigma^2)$, with $\boldsymbol{\ell}_t$ the logit row at position t, v^* its maximum, and σ

calibrated empirically. A worse predictor, whether from the integer model or the noise parameters, simply reports a larger $\hat{U}$, so tightening the bound is an engineering trade, not a soundness question (§10).

The bound is computed inside the proof from the LM-head logits by a few claims per output position (Appendix B.7). Its soundness property is the weaker downstream one of §7.2: the witness need not be unique, provided every prover freedom provably inflates the report.

5. Claim language

A model is written as ordinary tensor code against a tape, in the style of PyTorch. Each `tape.<op>` call records one claim, a public statement of what was computed, and returns a handle; handles overload the usual `@`, `*`, and `+`. A few lines of an attention block read:

```
norm = tape.rmsnorm(x, d=d, s=S, eps_int=EPS_INT)
g    = tape.hadamard_broadcast(norm, rms_w, SEQ=SEQ, d=d)
q    = tape.matmul(g, W_Q, s_a=S, s_b=S, s_out=S)
sc   = tape.matmul(qr, kr, transpose_b=True)   # attention scores
sm   = tape.softmax(sc, M=SEQ, s_x=S)
out  = tape.matmul(sm, v)
```

Each construction of §3.2 to §3.5, and the bound of §4, is one claim type. A claim does three jobs: it computes its values during witness generation, emits the linear and quadratic constraints that define a correct computation, and declares which of its values are committed with the forward pass and which only after a challenge (§6.2). The verifier recompiles the constraints from the public claim list and never accepts prover-supplied ones. Everything a claim depends on beyond the committed witness (lookup tables, quantization scales, noise-model parameters) is a public constant, fixed before any challenge. A model built from existing operations needs no prover or verifier changes; only a new operation needs a new claim type, which is how the surprisal claims of §4 were added. The remaining claim types (RMSNorm, SiLU, embedding select, elementwise operations, the routing weight) are compositions of the same machinery; their listings are Appendix B.

6. Implementation

We implemented the prover in CUDA and the verifier in Rust; the two share no code.

6.1 Ligero instantiation

VerInf works over the Goldilocks field, $|F| = 2^{64} - 2^{32} + 1$, which admits fast number-theoretic transforms and fits the fixed-point magnitudes of §3.6 without wraparound. The constants are

$$\mu = 8192, \quad \kappa = 16384, \quad \nu = 4 \cdot \kappa = 65536,$$

where μ is the number of message slots per row, κ the polynomial degree bound, and ν the codeword length, a Reed-Solomon rate of $1/4$. The hash is BLAKE3. The number of audited columns, τ, is a deployment choice that sets the soundness level (§7.3); the demonstrated runs are reported with their values in §9. The demonstrated runs hold the constants fixed as the witness grows, which is simpler but makes proof size and verifier work grow linearly rather than as the square root Ligero permits (§10).

A commitment (§3.1) is built as follows. The values are laid out as a matrix, each variable occupying a contiguous block of rows of μ slots. Each row is Reed-Solomon encoded: the μ message values, padded with $\kappa - \mu$ random slots for zero-knowledge, are interpolated to polynomial coefficients (an inverse NTT of length κ) and evaluated at ν points (a forward NTT) to give the codeword. The codeword columns are hashed into a Merkle tree whose single root is the commitment, and the audited locations of §6.2 are columns, revealed with their Merkle paths. One proof carries up to five such blocks under a single column-query set: the blinding rows, the weights R_W (committed once and persistent, §6.4), the refreshed weight block during a refresh, the forward-pass witness R_{p1}, and the challenge-dependent auxiliaries R_{p2}.

The folded test results of §6.2 are three short polynomials. The linear test aggregates the entire linear system into one weighted combination, Freivalds' check applied to the constraint system itself; the Hadamard test aggregates all the pointwise products; and the commitment-consistency test, an interleaved Reed-Solomon test ($\varepsilon_{\text{cons}}$ in §7.3), shows

that every row is close to a codeword. Soundness comes from Reed-Solomon distance: any inconsistency appears in a constant fraction of columns, so a few audited columns catch it with high probability (§7.3). The verifier checks the linear total against the public right-hand sides and the Hadamard total against zero before any column is audited; the audited columns then tie all three polynomials to the commitments.

Zero knowledge rests on two mechanisms. The random padding makes the values revealed at audited columns uniform conditioned on the message, provided the number of distinct columns ever audited against a commitment stays below $\kappa - \mu$; per-proof blocks refresh their padding every proof, and the persistent weight block's budget is managed in §6.4. Each test polynomial additionally mixes in a blinding row that leaves the verifier's checks unaffected while hiding its witness-derived part. Challenges are never materialized: each is a value of a hash PRF on (seed, index, label), derivable in $O(1)$ by both sides, with labels separating the challenge families.

6.2 Four round protocol

The checks of §3.2 to §3.5 use challenges of their own, drawn only after the initial commitments: the Freivalds projections after the matrices and their product are committed, the lookup challenges after the queries and multiplicities. Each check then commits new values built from its challenges (the projections, masked products, and lookup inverses). VerInf therefore inserts a commitment round between these challenges and the test challenges, so that the new values are fixed before the tests that fold them are drawn and cannot be fitted to a fold the prover has seen. This gives four rounds:

Round	Prover sends	Verifier replies
1	Commitments to the first-phase blocks: forward-pass witness (activations, routing masks, normalization auxiliaries, lookup multiplicities), weights (or a reference to a standing weight commitment, plus the refreshed block during a refresh, §6.4), and blinding randomness (§6.1)	Challenge seed s_{op}, from which the Freivalds projections ρ, λ and lookup challenges α, β derive
2	Commitment to the challenge-dependent auxiliaries (Freivalds projections, lookup inverses)	Test-challenge seed s_{comb}
3	Folded test results: linear, Hadamard, and a commitment-consistency test (§6.1)	Audit seed s_{col}, sent after first checking the parts of the test results that need no audit
4	The audited locations, derived from s_{col} by both sides, with the data to check them against the commitments	Accept or reject, after checking the audited locations against the commitments and the test results

Each seed is drawn only after the message above it is committed, so the per-challenge soundness bound (§7.3) holds per live attempt, with no offline search over challenges.

6.3 Streaming prover

The prover's field arithmetic, Reed-Solomon transforms, and Merkle hashing are GPU kernels over the Goldilocks field, compiled for the local device on first use.

At frontier scale the committed witness is far larger than any single machine's memory: for §9's 400B-parameter run it is about 7.2 terabytes (9.0×10^{11} field elements, Appendix A). The prover therefore streams the witness, committing one operation at a time: each row is encoded, folded into the column hashes and into running accumulators for the test polynomials, then freed before the next is computed. Because the Merkle tree is built over columns and the test polynomials are accumulations over rows, no later step needs the full encoded matrix resident, so peak memory tracks the working set of a single operation rather than the witness or the proof. Each round of §6.2 is one streaming sweep, with the witness regenerated per round rather than stored. Streaming bounds working-set memory only; proof size and proving time are unaffected. The same row-by-row structure admits parallelization across GPUs, with transform work split across rows and the column hashes partitioned across nodes, each accumulating its assigned columns (§10).

Peak memory is set by the largest single working set. At long context that is softmax's: its witness grows quadratically with context length, and the implementation proves each softmax over its full score matrix at once. This is a choice rather than a necessity, since the rows could be split into chunks and proven piece by piece, capping the working set at the chunk size (§10). One small piece of state persists across the whole proof: the lookup multiplicities of §3.4, kept as one resident histogram per table and fixed in size by the tables. Appendix C specifies how the constraint system is regenerated and evaluated against the streamed witness during the test folds; the resulting cost profile is analyzed in §8.

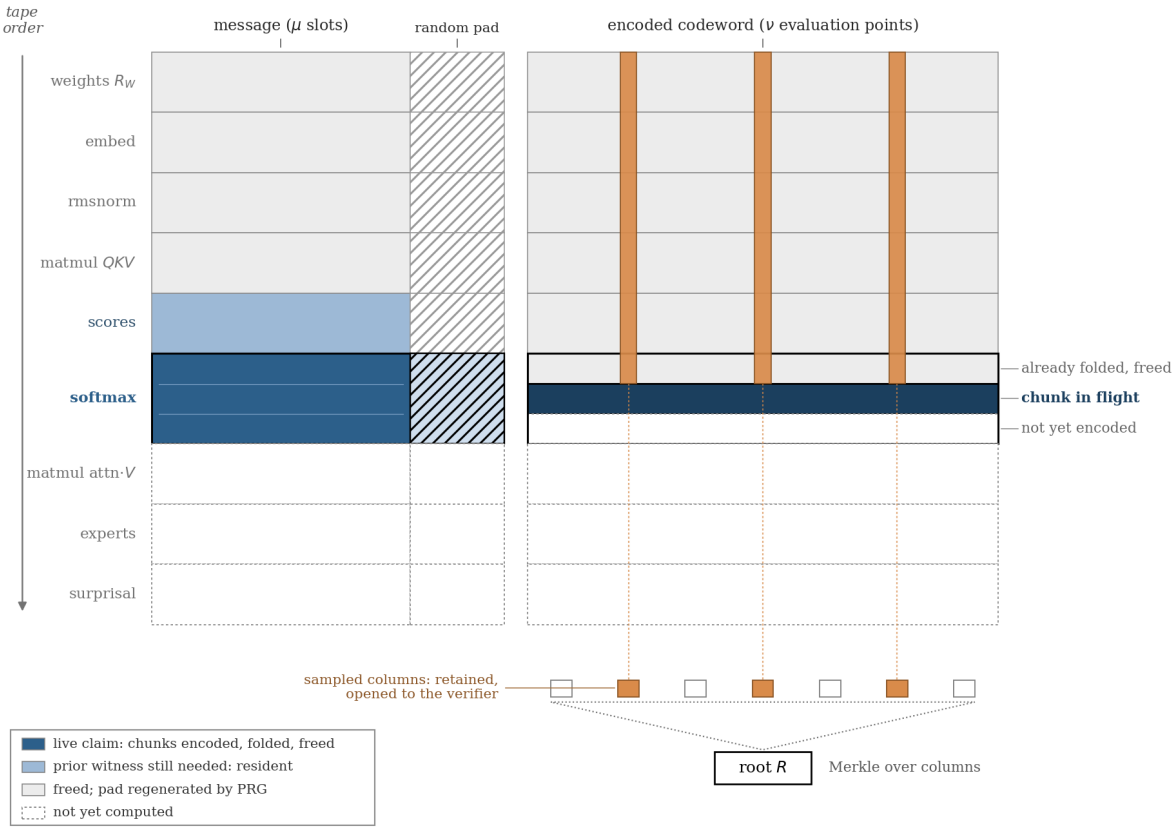

Figure 1: The streaming prover (§6.1, §6.3). Claims are committed in tape order: each row is encoded (left: message slots and zero-knowledge pad; right: codeword), folded into the column hashes and test-polynomial accumulators, then freed. Only the chunk in flight and prior witness still referenced stay resident. The Merkle tree is built over codeword columns; the audited columns are retained and opened to the verifier.

6.4 Weight commitment and refresh

The weights are committed once, and later proofs reference the standing root R_W rather than re-hashing the weight block, which removes the largest per-proof commitment cost at small context (§8).

Auditing spends hiding. Opening a commitment at columns C_1 in one proof and C_2 in another reveals exactly the evaluations at $C_1 \cup C_2$, so the single-proof hiding bound applies to the union: $\kappa - \mu = 8{,}192$ distinct columns are perfectly hiding, and the budget is the count of distinct columns ever audited against R_W across all proofs, on the order of a hundred proofs at production audit counts. τ sets only the spend rate; the per-test blinding rows are refreshed each proof and spend nothing.

When the union nears the budget, the prover refreshes: the same weights are re-committed under fresh randomness, giving a new root R'_W, and a linking proof ties it to the old one. The linking proof is a per-slot equality claim between the two weight blocks, checked by the existing linear test with no new machinery; a single differing element is caught with probability $1 - 1/|F|$, independent of how many columns are audited. The verifier adopts R'_W only if the proof accepts, the old block matches the currently trusted R_W, and the verified claim set is exactly the canonical link statement, so a prover cannot substitute a weaker relation. The bookkeeping that tracks the audited-column budget and triggers the refresh is future work (§10).

6.5 Verifier

The verifier is a Rust program that reads a proof and decides whether to accept it. It recompiles the constraint system from the public claim list (§5) and checks the proof against its own derivation, never against constraints supplied with the proof. Its work is, first, to confirm that the claim list meets the requirements of §7.2, and then to check that the audited columns re-hash to the committed roots and that the three test polynomials, recomputed at those columns, match the prover's. A proof is accepted only if every check passes.

The trusted base comprises the field arithmetic, the hash and Merkle check, the challenge derivation, the constraint compile, and these checks. Proof parsing and all other handling sit outside it, since a malformed or dishonest value fails a check and the proof is rejected. The verifier depends on three crates (`blake3`, `rayon`, `serde_json`) and is differential-tested bit for bit against a Python reference implementation. It needs no GPU, but at full model scale it needs a large-memory host, because the compiled constraint system grows with the witness; the heaviest step, the linear identity at the audited columns, parallelizes across cores and is the natural candidate for a GPU port (§10).

This division of labor follows the trust model of §7.1: only the verifier requires review, and prover-side code can be modified freely without enlarging the trusted base, since a fault there causes a proof to fail rather than to verify falsely.

7. Soundness

7.1 Trust model

The two parties want different guarantees. The prover wants confidentiality: the model weights, the activations, and the input and output token streams must not leak. The verifier wants soundness: the reported $\hat{U}$ must be a genuine upper bound for the committed tokens under the committed weights. Their interface is the public claim list, a statement of what kind of computation was performed (§3); it reveals the model architecture, though not the weights, and hiding the architecture as well is future work (§10). The proof reveals nothing beyond $\hat{U}$ and the claim list, so a dishonest verifier learns nothing more, and a dishonest prover cannot produce an accepting proof for a deflated bound except with the error bound of §7.3. Each side trusts only its own code: the verifier's trusted base is short (§6.5), and a fault anywhere on the prover's side can only cause a proof to fail, never to falsely verify.

7.2 The certified statement

What the proof certifies is a single statement over the committed witness, a conjunction of three parts:

$$\underbrace{H(\mathrm{AES}(\mathrm{key}, \boldsymbol{x} \parallel \boldsymbol{o})) = H_1}_{\text{transcript anchor (Appendix E)}} \;\wedge\; \underbrace{\boldsymbol{\ell}_t = D(\boldsymbol{x},\ \boldsymbol{o}_{<t})}_{\text{forward pass (B.2 to B.6)}} \;\wedge\; \underbrace{-\textstyle\sum_t \log_2 Q(\boldsymbol{o}_t \mid \boldsymbol{\ell}_t) \;\le\; \hat{U}}_{\text{surprisal bound (B.7)}}$$

with $\boldsymbol{x}$ and $\boldsymbol{o}$ the committed input and output token streams, H_1 the digests recorded at generation time, one per stream (alongside the key commitment H_2), D the committed integer model of §3, $\boldsymbol{\ell}_t$ the logit row scoring position t, Q the committed predictor of §4, and $\hat{U}$ the reported value. The parts are held to different standards. The anchor and the forward pass must each leave the prover no freedom at all, the first by collision resistance and the second by constraint structure; the surprisal computation may leave freedom, provided every free direction inflates the reported value.

Transcript anchor. The bound is conditioned on the input and scored on the output, so it is meaningful only when both token streams are the ones the run actually used; inside the proof they are hidden witness, which on its own ties them to no external record. The anchor binds them to digests recorded independently at generation time: a recorder hashes the encrypted token streams as they pass and receives a commitment to the key material, fixed with the request before the response exists, and the proof shows that the committed tokens encrypt and hash to the recorded digests, without revealing them. With the ciphertext and the key both fixed, the token stream is the unique decryption, so uniqueness here comes from collision resistance rather than from constraint structure (Appendix E). The record itself must come from a process the verifier trusts, independently of and prior to the proof; in the setting of §1 this is the verifier's network-boundary hardware (Petrie et al. 2025). The circuits for the anchor are implemented and tested; demonstrating them end to end against recorded digests is future work (§9).

Forward pass. Upstream of the logits, every claim must admit exactly one satisfying assignment: slack in an intermediate value propagates through the remaining layers in directions that cannot be analyzed, so any freedom there could cascade into the token probabilities arbitrarily. The construction meets this claim by claim (Appendix B.2 to B.6). Softmax is where the requirement trades against efficiency: a tolerance band on the normalization admits several shifts, each a direct lever on the reported bound, so VerInf pins the shift to a single integer instead (§3.5, Appendix B.3). A

second requirement is causality: the prediction of each output token must depend only on earlier tokens, so a later token cannot be used to lower the surprisal of an earlier one. This is enforced by the public attention mask compiled into the claims.

Surprisal bound. The inequality itself is defined in §4: the reported $\hat{U}$ must be at least the surprisal sum for the transcript actually produced, realization by realization. Downstream of the logits, in the short computation from logits to $\hat{U}$, freedom is therefore permitted provided every free direction increases the reported value, a weaker property than upstream uniqueness that can be established directly because the downstream computation is a few steps of analyzable arithmetic. The construction meets it by pushing every rounding in the surprisal computation upward (Appendix B.7).

The last two parts are properties of the claim graph, which is public and recorded with the proof. The verifier checks the proof against this claim list; that the claim list itself has the required structure, for example that each weight is read only in a forward pass and never updated so that no gradient step is hidden in the computation, is established by auditing it. Confirming this automatically, by static analysis of the claim graph, is future work (§10).

7.3 Protocol-level soundness

Every claim carries a soundness lemma in Appendix B discharging the requirement of §7.2: values defined by constraints from values above them are unique by construction, and every declared value must be pinned, uniquely upstream of the logits, or, downstream in the surprisal claims (§4), with every freedom shown to inflate the reported bound. A declaration without an argued pin is a defect by definition.

At the protocol level, the probability that a cheating prover is accepted is bounded by the sum of the test errors and the per-claim reduction errors:

$$\varepsilon \;\le\; \underbrace{\varepsilon_{\text{cons}} + \varepsilon_{\text{lin}} + \varepsilon_{\text{quad}} + \varepsilon_{\text{field}}}_{\text{argument}} \;+\; \sum_{\text{matmuls}} \frac{2}{|F|} \;+\; \sum_{\text{tables}} \frac{M + T_{\text{len}} + 1}{|F|},$$

with range checks contributing nothing (word decompositions are exact). At the parameters of §6.1 the test errors are $\varepsilon_{\text{cons}} = (3/4)^\tau$, $\varepsilon_{\text{lin}} = (3/8)^\tau$, and $\varepsilon_{\text{quad}} = (1/2)^\tau$ in the audit count, so the consistency term dominates and reaching 2^{-s} needs $\tau \approx 2.4\ s$; $\varepsilon_{\text{field}} \approx 2^{-48}$, and the Freivalds terms are negligible at $|F| \approx 2^{64}$. The binding term in practice is the LogUp sum: the busiest tables absorb about 7×10^{10} queries at frontier scale, a per-instance error near 2^{-28} (B.1.0), tightened where needed by parallel repetition of the lookup challenge.

Both the audit count and the Ligero constants are parameters of the demonstrated implementation, not of the protocol. τ is the dial: more audits drive the consistency term down geometrically until, past roughly 80, the LogUp term binds and parallel repetition is also required; the demonstrated values are reported with the runs (§9). The row width and degree are equally free: the implementation holds μ and κ fixed as the witness grows, which is simple but makes proof size and verifier work grow linearly, and a deployment would instead grow the row width with the square root of the witness, recovering the $\sqrt{W}$ scaling Ligero permits (§6.1, §10). A modest per-challenge level is meaningful in the setting of §1: the protocol is interactive, so the bound holds per live attempt with no grinding, and the intended parties are deterred by any non-negligible chance of being caught even once, since a detected violation is diplomatically costly.

8. Cost and scaling

The prover's cost is governed by three quantities, each a polynomial in the context length S: the committed witness size W, the number of distinct linear constraints L, and the number of quadratic products Q. The accounting is per claim: each claim type contributes to the three drivers as a closed-form function of its shape parameters, read off its listing in Appendix B. A rescaled matmul of shape (m, k, n), for example, commits $6mn + 3k$ witness slots, the raw product and its five rescale words accounting for the $6mn$ and the three length-k Freivalds vectors for the $3k$; softmax commits 15 slots and 8 quadratic products per score cell. Appendix A.1 tabulates these footprints for every claim type. Instantiating them at Llama-4-Maverick's dimensions and summing over its claim list, with each term constant, linear, or quadratic in S, gives

$$\begin{aligned} W(S) &\approx 4.00{\times}10^{11} + 4.48{\times}10^{8}\ S + 40320\ S^2, \\ L(S) &\approx 1.19{\times}10^{8} + 1.50{\times}10^{8}\ S + 12480\ S^2, \\ Q(S) &\approx 5.93{\times}10^{7} + 1.54{\times}10^{8}\ S + 19200\ S^2. \end{aligned}$$

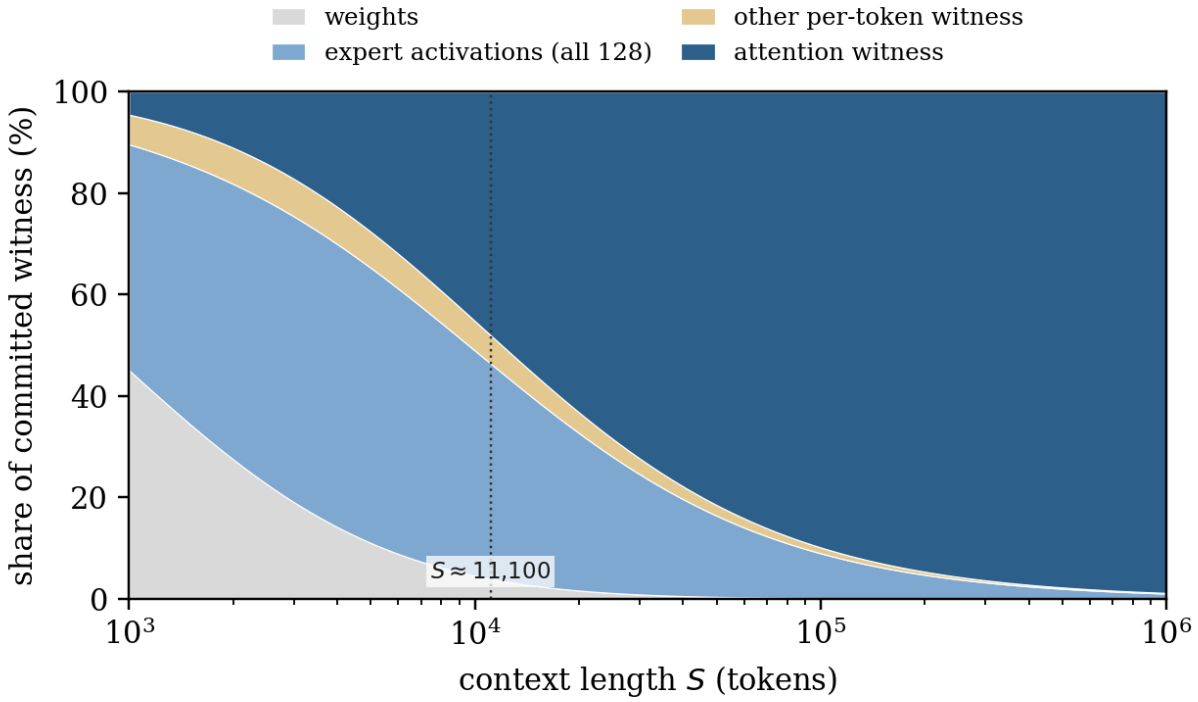

Figure 2: Composition of the committed witness for Llama-4-Maverick as context grows (Appendix A.4). Weights dominate below about 900 tokens, the committed experts from there to the second crossover at $S \approx 11{,}100$, and attention above.

The totals are validated at full scale: the verifier's independent compile of the demonstrated runs (§9) yields witness and quadratic row counts within about 1% of the model at both $S = 1000$ and $S = 1093$ (Appendix A.2).

Both leading coefficients have closed forms, so the cost can be understood from two claim types. The S^2 term is attention exactly: the scores matmul and softmax are the only claims with a quadratic term, committing 21 witness slots per score cell, and $21\ n_q\ n_{\text{layers}} = 40320$ (Appendix A.3). The S term is dominated, about 89%, by the mixture-of-experts matmuls, committed for all $E = 128$ experts per MoE layer even though one fires (§3.3), about $4{\times}10^8$ slots per token. The constant term of W is the committed weights; L and Q have almost none, because Freivalds compresses each weight matmul to $O(k)$ constraints (§3.2).

The three terms dominate the witness in turn as context grows: the weights below about 900 tokens, the committed experts from there to about 11,100, and attention above (Figure 2; the tabulated shares are in Appendix A.4).

The demonstrated run sits at the first crossover, its witness about 90% weights and committed experts; at frontier context the witness is almost entirely attention and the cost can be sized from $W \approx 40320\ S^2$ alone.

Wall-clock time follows from these drivers through a per-primitive cost identity (Appendix A.5), with each heavy step priced by a measured hardware rate; the transforms are memory-bandwidth bound, so the identity's leading terms ride bandwidth, except the column hashing, which rides hash throughput. The four-round protocol multiplies only the witness recomputation, the cheapest term. At $S = 1000$ the identity gives a floor of roughly 8 to 10 hours against the measured 14.3 hours (§9): the implementation is within about 1.5x of its own floor, with the gap in the fold's remaining memory traffic and orchestration (Appendix C), not in the protocol. (The pre-optimization run measured 19.3 hours at $S = 1093$, about 2x the floor; the difference is the fold work of Appendix C landing at full scale.)

At a context of one million tokens the same identity prices a dense-attention proof at roughly 25 years on the demonstrated machine. The bandwidth-riding terms divide by a cluster's aggregate-bandwidth ratio, on the order of 2,500 for an NVL72-class machine, bringing them to days; the column hashing divides by hash throughput instead and joins the leading order at that scale [confirm: per-GPU BLAKE3 rate], which is one motivation for the hash and verifier engineering of §10. Two structural reductions are available. The projection assumes dense attention in every layer: a

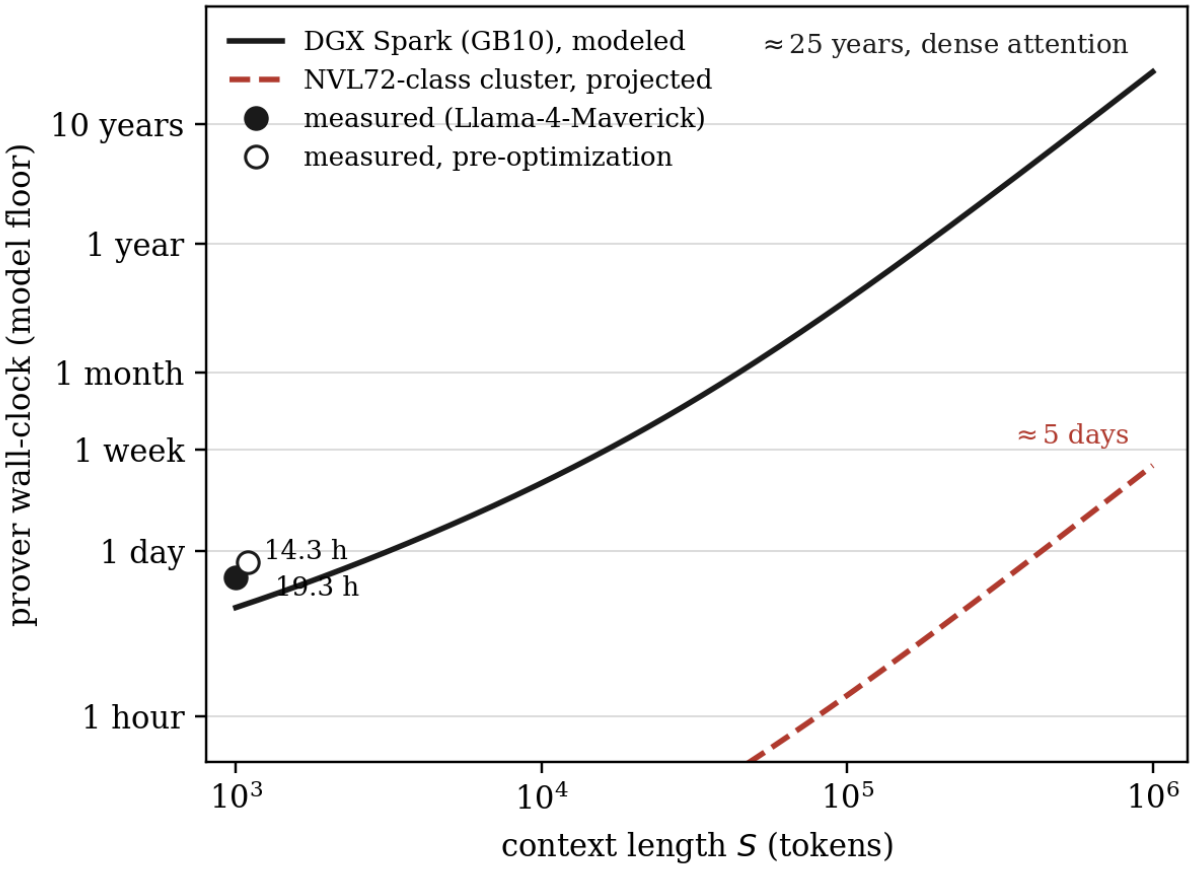

Figure 3: Prover wall-clock against context length: the modeled floor from the cost identity of Appendix A.5 on the demonstrated machine (solid), the measured runs of §9 (points), and the projection to an NVL72-class cluster (dashed), which divides the bandwidth-riding terms by the aggregate-bandwidth ratio while the column hashing rides hash throughput. Both curves assume dense attention in every layer.

model whose layers are all windowed or sparse cuts the attention term by the ratio of context to window, after which the committed-experts term is the floor, an overall reduction of one to two orders of magnitude; a model that interleaves, keeping dense attention in a fraction of layers, cuts the quadratic term only by that fraction. And peak memory at long context, set by the softmax residency, is removable by chunking (§6.3) independently of these witness reductions.

The remaining prover levers are the streaming schedule and the argument itself. Proving claims one at a time rather than in four passes removes the repeated witness sweeps and the column-opening re-encode, worth roughly a third of the floor (Appendix A.5); beyond that, a larger speedup requires changing the argument, for example replacing Ligero's linear test with a sumcheck- or GKR-style protocol.

9. Results

We report two runs, both produced by the streaming prover of §6.3 on a single NVIDIA DGX Spark (GB10, 121 GB unified memory) and checked by the verifier of §6.5, which shares no code with the prover.

Llama-4-Maverick, 1000 tokens, every token hidden. The full 48-layer, 400B-parameter mixture of experts, with all 128 experts committed per MoE layer, proven in the four-round protocol of §6.2 with 40 columns opened. The transcript is a 500-token prompt and the model's own 500-token greedy continuation; all 1000 tokens are hidden, entering the proof only as committed one-hot indicators, and the indicator rows are shared between the input selection and the surprisal claims of §4, so the scored tokens are, by shared committed variable, the tokens the model consumed. The

surprisal claims run inside the proof and the bound is its only public value: $\hat{U} = 0.880$ bits per token over the 500-token continuation, explaining about 95% of the information a token from the 202,048-token vocabulary can carry. The proof took 14.3 hours to generate at a prover peak of 78.1 GB GPU memory (83.9 GB unified); the committed witness is about 7.2 terabytes, streamed at the working set (§6.3); Figure 3 places the measured runs on the modeled curve. The verifier accepted the 93.6 GB proof, checking all 40 opened columns, in 17.7 hours on 20 CPU cores at a peak of 75.7 GB; the per-challenge soundness bound is about $2^{-16.6}$ (§7.3). An earlier 1093-token run (19.3 hours, $\hat{U} = 0.394$ bits per token, continuation tokens public) predated two prover soundness fixes (a vacuous RMSNorm bracket and a constraint-fold defect) and is superseded by this run; its lower bound reflects a transcript generated by a different backend with higher integer-model agreement, a predictor-side difference that §4 prices, not a change in the proof system. Raising the configuration to 80 opened columns is a deployment choice whose measured cost is verification runtime, since the per-column work grows with the count (a GPU verifier is the identified path, §10); verifier memory is dominated by parsing the opened columns and does not depend on how many are checked.

Llama-2-7B, 1000 tokens. All 32 layers on real checkpoint weights: the forward-pass proof generates in about 44 minutes at a prover peak of 11.2 GB with the weights streamed from disk, producing a 1.44 GB proof at 10 opened columns that the verifier accepts in about 23 minutes on 20 CPU cores.

Token binding. The runs above do not include the transcript binding of Appendix E. Both committed token streams are hidden and internally consistent: the input tokens and the scored output tokens are one committed stream, so the bound is certified over the transcript the proven forward pass actually consumed. What remains open is anchoring that committed transcript to a record produced at generation time (§7.2): the AES and SHA-256 circuits of Appendix E are implemented and tested claim types, and demonstrating the binding end to end against recorded digests is future work (§10).

Negative controls. Tampered proofs are rejected: a modified opened column, a modified test polynomial, and a cheating routing witness each fail verification. A claim-level negative suite applies one targeted tamper per verifier check and passes on every claim type. **[TODO: add the Appendix B.7 negative test to the suite and report it: with the paired lookup binding $\mathrm{Pow}[b]$ removed, a deflated bound must be accepted, demonstrating that the lookup is load-bearing; in the shipped configuration the same tamper must be rejected.]**

The cost model of §8 was fitted to measured hardware primitives and checked against these runs (Appendix A.2); a companion benchmarks document tabulating predicted against measured times is planned.

10. Limitations and future work

VerInf is a research prototype and has not had a security review. The demonstrated system has four main caveats. The proofs are large, gigabytes at full model scale (§9). The demonstrated run leaves 0.880 bits per token unexplained, which may not suffice for every application. The soundness of a demonstrated configuration is a per-challenge bound (§7.3), adequate in the setting of §1 but a deployment choice rather than a fixed property. And the public claim list reveals the model architecture (§7.1). The main directions for future work are:

Tightening the bound.

- Commit the low-precision floating-point intermediates directly, rather than an Int64 approximation, and prove a similarity claim per operation, so quantization error does not propagate through the proof; proving exact low-precision integer inference is a natural first step. Where a deployment runs deterministic kernels, modeling them exactly in the proof is the limiting case of the same direction, driving the bound toward zero; bitwise-reproducible inference has been demonstrated across GPU variants (Cankaya 2026), and a deployment that adopts it needs only this limiting case.
- Tune the quantization scales, table sizes, and noise-model parameters, which trade unexplained information against proving time; §4 prices every such choice through the bound.

Prover cost.

- Reduce the committed intermediates that set the leading cost (§8): the quadratic attention witness and the mixture-of-experts commitment.
- Chunk the softmax witness to remove the one large residency (§6.3), decoupling peak memory from context length.
- Prove claims one at a time rather than in four passes, worth roughly a third of the cost floor (Appendix A.5), and parallelize the prover across GPUs along the row structure of §6.3.

Proof size and verification.

- Port the verifier's column checks to a GPU, keeping the CPU version as the bit-exact reference; verification runtime is the measured cost of higher soundness levels (§9).
- Reduce proof size with a wider column format, and grow the polynomial degree with the witness so that proof size and verifier work scale as the square root of the witness, as Ligero permits; the demonstrated runs hold the degree fixed, which is simpler but makes the proof grow linearly.
- A budget ledger for the persistent weight commitment: the refresh, and the linear proof linking an old commitment to its replacement, are implemented and tested (§6.4); the remaining piece is the prover-side bookkeeping that tracks the opened-column budget and triggers the refresh.

Assurance.

- Formally verify the two properties of §7.2: that the forward-pass claims admit unique witnesses, including a systematic magnitude and width audit of every bracket and exclusion operand (§3.6), with the surprisal claims of Appendix B.7 first; and that the causal mask prevents a later token from influencing an earlier one.
- Replace the manual audit of the claim graph with automated analysis, for example confirming that each weight is read only in a forward pass so that no gradient step is hidden in the computation.
- Hide the model architecture with a second proof stage showing that the verifier's own check ran and accepted (§7.1), so architecture-dependent conditions never surface.
- A security review of the construction and the claim set.

The same claim types support further model architectures with no change to the prover or verifier (§5), and demonstrating the token binding of Appendix E at full scale, against digests from real recording hardware, would complete the chain from network capture to certified bound (§7.2, §9).

Appendix A. Cost model

This appendix derives the cost figures of §8: the per-claim contributions to the three drivers W (committed witness slots), L (distinct linear constraints), and Q (quadratic products); their sums over Llama-4-Maverick as polynomials in the context length S; the closed forms for the leading terms; and the machine constants that turn (W, L, Q) into wall-clock.

A.1 Per-claim contributions

Each claim type contributes to W, L, and Q as a function of its size parameters. The forms below are for the Maverick configuration, in which every matmul, hadamard, rope, and rmsnorm carries an output-rescale block (marked ⓡ) and softmax runs in its saturating-causal form. Matmul shapes are written (m, k, n, H) for an H-head batch of $(m, k) \times (k, n)$ products, with $mHn \equiv m\ H\ n$; elementwise claims have length N; routing and combine are over T tokens, E experts, feature width F.

claim	W	L	Q
matmul (m, k, n, H) ⓡ	$6\,mHn + 3k$	$2k + H + 2\,mHn$	$k + 2\,mHn$
rmsnorm (B, d) ⓡ	$7Bd + 82B$	$17B + 2Bd$	$3Bd + 42B$
softmax (B, M) sat+causal	$15BM + 9B$	$\frac{1}{2}B(M{+}1) + 4BM + 5B$	$8BM + 3B$
silu (N)	$23N$	$7N$	$12N$
hadamard (N) ⓡ	$6N$	$2N$	$3N$
rope (N) ⓡ	$6N$	$3N$	$2N$
add (N)	N	N	0
paired lookup (N)	$3N$	N	N
routing (T, E)	$10TE + 2T$	$3TE + 3T$	$5TE$
masked-combine (T, E, F)	$2ETF + TF$	$ETF + TF$	ETF
freivalds-combine (T, E, F)	$TF + 4ET + T$	$3ET + 2T$	ET

The rescale term in the matmul row, $2\,mHn$ in both L and Q, is the output-rescale block; it is what makes a rescaled matmul cost $O(mn)$ constraints rather than $O(k)$, and so determines whether the weights are effectively free in the constraint counts. Attention specializes these claims: the scores $QK^\top$ are a matmul (S, d, S, n_q), an $n_q S^2$ output; softmax is $(B{=}n_q S, M{=}S)$; the value product is a matmul $(S, n_q S, d_h, n_q)$, an $O(S)$ output. In these attention shapes the inner dimension k is written summed over heads ($k = n_q d_h = d$ for the scores), so the $O(k)$ Freivalds terms count all heads at once; the per-head claims total identically. The expert sums use freivalds-combine ($\approx 4ET$) rather than masked-combine ($\approx 2ETF$).

A.2 Summed totals

Summing the per-claim contributions over the 48-layer model (24 dense, 24 MoE; $d = 5120$, $d_{\text{ff,exp}} = 8192$, $E = 128$ top-1 with all experts committed, $n_q = 40$, $V = 202048$) gives the totals quoted in §8:

$$
\begin{aligned}
W(S) &\approx 4.00{\times}10^{11} + 4.48{\times}10^{8}\ S + 40320\ S^2,\\
L(S) &\approx 1.19{\times}10^{8} + 1.50{\times}10^{8}\ S + 12480\ S^2,\\
Q(S) &\approx 5.93{\times}10^{7} + 1.54{\times}10^{8}\ S + 19200\ S^2.
\end{aligned}
$$

The matmul claims dominate the linear (S) coefficient of all three: the QKVO projections, the 128 expert matmuls per MoE layer, the FFN, and the LM head. Softmax, together with the scores matmul, dominates the S^2 coefficient. The constant term of W is the committed weights, taken as the parameter count $4{\times}10^{11}$; L and Q have only a small constant from the Freivalds auxiliaries, because Freivalds compresses each weight matmul to $O(k)$ constraints.

The totals are validated at full scale by the demonstrated runs. The verifier's independent compile of the 1093-token proof reports 115,235,029 witness rows and 23,554,246 quadratic rows; at $\mu = 8192$ slots per row that is $W = 9.44{\times}10^{11}$ and $Q = 1.93{\times}10^{11}$, within 1% of the model's $W(1093) = 9.38{\times}10^{11}$ and $Q(1093) = 1.91{\times}10^{11}$. The 1000-token all-hidden run of §9 checks the same way: 109,267,016 witness rows and 21,370,360 quadratic rows, $W = 8.95{\times}10^{11}$ and $Q = 1.75{\times}10^{11}$ against $W(1000) = 8.88{\times}10^{11}$ and $Q(1000) = 1.73{\times}10^{11}$, again within about

1% (the hidden-token indicator claims it adds sit below the percent level). The S^2 coefficient is separately anchored by measurement: single-block runs across context lengths fit 840 committed slots per block per S^2, identical at $E = 8$ and $E = 128$ (confirming the quadratic term is attention alone), and $840 \times 48 = 40320$.

A.3 Leading terms

Both leading coefficients have closed forms, so the cost can be understood from two claim types.

The S^2 term is attention. Softmax and the scores matmul are the only claims with an S^2 term, and each layer forms an $S \times S$ score matrix for each of n_q heads, so $n_q S^2$ score cells per layer. Per cell, the scores matmul commits 6 values (the score, its raw product, the two rescale words, and two range-check inverses) and softmax commits 15 (the two exponential-table lookups, their LogUp inverses, the paired-lookup combinations, and the saturating mux), giving $(6 + 15)\ n_q\ n_{\text{layers}} = 21 \times 40 \times 48 = 40320$ witness slots per S^2. Counting constraints and quadratic products per cell instead gives the L and Q coefficients:

per score cell	scores matmul	softmax	total	$\times\ n_q\ n_{\text{layers}}$
W	6	15	21	40320
L	2	$4\frac{1}{2}$	$6\frac{1}{2}$	12480
Q	2	8	10	19200

The half in the softmax L count is the causal mask: only the lower triangle of each $S \times S$ block is constrained, so that constraint family is $\frac{1}{2}\ n_q S^2$.

The S term is the committed experts. Each MoE layer runs all E experts, committed even though one fires (§3.3). Each expert is three matmuls on the shared S-token input with output sizes $S\ d_{\text{ff,exp}}$, $S\ d_{\text{ff,exp}}$, and $S\ d$, each carrying the $6\times$ rescale block, so $6\ S\ (2d_{\text{ff,exp}} + d)$ per expert, and over E experts and $n_{\text{moe}} = 24$ layers,

$$6\ E\ (2d_{\text{ff,exp}} + d)\ n_{\text{moe}} = 6 \times 128 \times 21504 \times 24 \approx 3.96{\times}10^8$$

slots per token, about 89% of the linear coefficient; the remainder is the attention projections, the nonlinearities, the norms, and the LM head. The constant term of W is the committed weights.

A.4 Which term dominates

The three terms dominate W in turn as context grows: weights below about 900 tokens, the committed experts (linear) from there to about 11,100, and attention (quadratic) above. The demonstrated 1000-token run sits at the first crossover.

S	W	weights	experts	attention
1,000	$8.9{\times}10^{11}$	45%	45%	5%
4,000	$2.8{\times}10^{12}$	14%	56%	23%
11,100	$1.0{\times}10^{13}$	4%	43%	48%
100,000	$4.5{\times}10^{14}$	0%	9%	90%
1,000,000	$4.1{\times}10^{16}$	0%	1%	99%

Shares do not sum to 100%: the remainder is the non-expert linear work (attention projections, norms, nonlinearities, the LM head), about 6% at the low end. The 11,100-token row is the second crossover, where the linear and quadratic terms are equal by construction. At small context the proof is model-dominated, with weights and committed experts about 90% of the witness; at frontier context it is almost entirely attention, and the cost can be sized from $W \approx 40320\ S^2$ alone.

A.5 Wall-clock from (W, L, Q): pass accounting

The streaming prover never stores the witness (the encoded matrix at $S = 1000$ is 57 TB), so the four-round protocol recomputes the witness values once per round. That recomputation is the *only* cost the rounds multiply; every heavy step runs once. The cost identity is

$$T \approx 4\,T_{\text{wit}} + (A_c + A_f + A_x)\,W + D\,W + E\,W + C\,Q + B\,L + T_{\text{aux}},$$

with each term priced by a measured GB10 primitive (Reed-Solomon NTT: 0.42 ns per element at length 2^{15}, memory-bandwidth bound at the measured 223 GB/s; a faster NTT kernel was measured to have no headroom, so the only transform lever is fewer transforms):

- T_{wit}: one witness-computation sweep (the integer forward pass and derived values), compute-bound, about an hour per pass at $S = 1000$. It is the only $\times 4$ term and the cheapest, and it stays negligible at every scale.
- $A_c \approx 4.2$ ns/slot, the commit encode: an inverse NTT of length κ plus a forward NTT of length ν is ten transform elements per slot.
- $A_f \approx 3.4$ ns/slot, the linear fold's own transforms (two per row with the fused inverse NTT, Appendix C). These are not a redundant re-encode: the coefficient polynomial does not exist until the round-3 challenges arrive.
- $A_x \approx 4.2$ ns/slot, the column-opening re-encode in round 4. Producing the τ opened columns by one more full encode is cheaper than direct evaluation at τ points once $\tau \gtrsim 10$. No re-hashing is required: the column-hash tree from round 1 is a few megabytes and is kept (the implementation re-derives the root anyway as a cheap cross-round consistency check).
- D, the column hashing: the encoded matrix is $8W$ elements and BLAKE3 absorbs eight per compression, so one compression per witness slot. The measured primitive is 2.0 gigacompressions per second (about 0.5 ns per slot), and instrumented runs of both claim mixes put the hashing bucket at 5 to 8% of proving time. This term is compute-bound, the one leading term that rides hash throughput rather than memory bandwidth.
- E, the constraint-coefficient work for the linear fold, proportional to the nonzeros of the constraint matrix, $O(W)$. The fold kernels compute each target slot's coefficients in registers directly from its band descriptor (Appendix C), one thread per slot with nothing materialized or sorted, so the term is arithmetic fused into the fold: 4 to 5% of proving time measured across both claim mixes.
- $C \approx 15$ ns per quadratic product (the products and re-encode of the quadratic fold, bandwidth-bound); $B \approx 0.6$ ns per linear constraint (one challenge hash), negligible at scale.

Calibration against the demonstrated runs: round 1 measured 10.3 ns per witness slot against its $A_c + D$ budget of about 5.5; the pre-fold-optimization run (19.3 hours at $S = 1093$) was within about $2\times$ of the identity's total of 8 to 10 hours, and the demonstrated run (14.3 hours at $S = 1000$, with the Appendix C fold optimizations landed) is within about $1.5\times$. Instrumented two-layer runs of both claim mixes (dense and mixture-of-experts; cuda-synced bucket shares) decompose the proving time into: witness 29%/10%, encode 28%/46%, quadratic fold 20%/10%, linear fold 14%/21% (of which the transforms are about two-thirds and the coefficient work a quarter), column hashing 5%/8%, and auxiliary commit 3%/6%. The residual against the identity sits in the fold's last DRAM round trip and chunk orchestration: implementation, not protocol. Collapsing the identity to a single per-slot transform constant underprices the same run by about $6\times$: the hash, coefficient, and witness terms are load-bearing, not overhead.

Scaling behavior differs by term. A_c, A_f, A_x, C ride aggregate memory bandwidth and divide by a cluster's bandwidth ratio (about 2,500$\times$ for a 72-GPU NVL-class machine). D rides hash compute: it divides by GPU count and per-GPU scalar-ALU rate, not bandwidth, and since a B200's scalar throughput is only about $2.4\times$ a GB10's, the hash term scales by roughly $170\times$ where the transforms scale by 2,580$\times$, leaving it a quarter to a third of the cluster floor at one million tokens. T_{wit} rides matmul compute and stays negligible. Claim-streaming (§10) removes three of the four witness passes and the A_x term by completing each claim's rounds while its rows are live, worth roughly a third of the floor rather than the $4\times$ a per-round count suggests. The projection assumes dense attention, which a sliding-window or sparse pattern replaces with an $O(Sw)$ term.

A.6 Not modeled

The weights floor is taken as the parameter count rather than recomputed from layer shapes (a few-percent refinement). The one-hot validity proof for hidden prompt tokens costs $O(V)$ slots per hidden token, about 0.1% of the per-token witness, and is omitted; it scales with the hidden-prompt length, not the full context. The unexplained-information machinery on the logits is not a per-layer arithmetic claim and is not counted.

Appendix B. Claim specifications

This appendix specifies every claim type. Each entry gives a listing of the claim's commitments and constraints in protocol order, the witness and constraint counts the listing contributes, a soundness lemma discharging every

declaration, and a note on how the concrete shapes generalize. The listings use a small fixed grammar, defined below, with two aims: a claim's totals can be read off its listing line by line, and every point where the prover has any freedom is visible as a declaration the lemma must argue. The listings match the constraint systems the implementation emits, and the counts match the cost model of Appendix A.1.

A single line shows most of the grammar at work:

$$\texttt{quad} \quad \boldsymbol{p}[i] \leftarrow \boldsymbol{u}[i]\ \boldsymbol{y}[i] \qquad \forall\, i \in [k]$$

Read: for each i, the prover commits $\boldsymbol{p}[i]$, defined as the product of the already-committed $\boldsymbol{u}[i]$ and $\boldsymbol{y}[i]$, enforced by one quadratic constraint. The keyword names the test the constraint feeds; the arrow marks $\boldsymbol{p}$ as newly defined, exactly one new variable per line; bold marks all three as committed witness; the third column gives the extent, the indices the line ranges over, bound with their ranges at first use. Each keyword has a fixed footprint of witness slots and constraints per element of its extent; the last three columns of every listing record the resulting contributions to W, L, and Q, so a claim's totals are column sums.

Typography. Bold marks the witness. Every committed value, whether inherited input, declaration, or arrow-defined, is set in bold ($\boldsymbol{c}$, $\boldsymbol{z}$, $\boldsymbol{e}_1$), and everything thin is public: indices and sizes, table names ($\mathrm{T_A}$, Exp), public constants ($\mathrm{Z_{max}}$, $\mathrm{s_y}$, δ), challenges (ρ, λ), and index predicates. Scanning any listing, the thin symbols are exactly what the verifier knows, and every bold symbol is a commitment. A lemma may introduce thin working symbols of its own (the error matrix E of B.2); bold is reserved for committed values. All committed values are Goldilocks field elements; listings do not repeat this. A value that is private until a final pin reveals it (the surprisal sum of B.7) is bold like any witness, with the revealing pin noted.

Line kinds.

- `input`: values inherited from upstream claims or the committed weights, with their extents. No slots or constraints; each value is counted by the claim that creates it, and an `input` line promises nothing about its values.
- `decl`: the prover asserts private variables, one slot per element, constrained by nothing at introduction. Every declaration must be argued in the lemma.
- `chal`: verifier randomness, with its extents. No slots and no constraints; challenges are derived from the round seed, not committed, and everything on a `chal` line is public.
- Arrow lines, tagged `lin` or `quad`: define the left side, exactly one new variable per line, from values above. One slot and one constraint of the tagged kind per element.
- == lines, tagged `lin` or `quad`: constrain existing values, one constraint and no slots. A == that pins a declaration is argued in the lemma; the marker $\leq$ denotes a deliberately one-sided pin.
- `range`: $\boldsymbol{x} \sqsubseteq \mathrm{NAME}$ pins $\boldsymbol{x}$ into the named registry table, one slot (the lookup inverse) and one quadratic per element. The signed form $\boldsymbol{x} \sqsubseteq \pm\mathrm{NAME}$ pins $\boldsymbol{x} \in [-2^{w-1}, 2^{w-1})$ by committing the shifted copy $\boldsymbol{x} + 2^{w-1}$ and checking it against the same width-w table: two slots, one linear, one quadratic. There is no separate signed table.
- `lookup`: $\boldsymbol{v} \leftarrow \mathrm{NAME}[\boldsymbol{key}]$ defines $\boldsymbol{v}$ as the table's value at $\boldsymbol{key}$ and simultaneously pins $\boldsymbol{key}$ into the table's key range, the one line with two jobs. Three slots (value, folded key, inverse), one linear, one quadratic per element; the expansion is the paired lookup of B.1, whose challenge-turn placement follows B.1 without appearing in listings.
- `rescale`: $\boldsymbol{x} \leftarrow \texttt{rescale}(\boldsymbol{x}_{\text{full}})$ returns a raw product to the working scale, keeping the signed high word and dropping the low bits: the third fixed-footprint composite. Five slots (the kept word, the low word, the shifted copy, two inverses), two linear, two quadratic per element; the expansion and its tables are in B.1.

Extents and binding. The third column carries each line's extent in $\forall$ notation. An index letter is bound with its range once, at the first line using it; later lines write the bare letter. An extent may be filtered by a public predicate on its indices, as in $\forall\, h, q,\ i \leq q$, and the Iverson bracket $[[\cdot]]$ writes such a predicate as a public 0/1 value inside an expression. When a definition's filter does not cover its variable's full extent, the remaining elements are unconstrained: they must appear as a `decl` over the complement, and the lemma must argue that freedom. A comment, when one is needed, sits on its own italic row above the line it annotates, and is used for declarations and phase boundaries only; a constraint line's equation is its own documentation.

Public constants and the registry. Every claim's size parameters and scales are public constants of the claim list; a claim's header names only its further public constants, such as window widths and saturation thresholds. Everything

about lookup tables lives in one place, the table registry (B.1.0): contents, rounding rule, length, which claims share the table, and total query volume. An upright name in a `lookup` bracket or after $\sqsubseteq$ is a key into the registry, and listings never restate registry facts. In the demonstrated configuration every claim's inputs and outputs sit at the working scale $\mathrm{S} = 2^{12}$, with each multiplicative claim rescaling its output to S (B.1); the compiler supports per-claim scales, and the surprisal claims use distinct scales for table values and nats (B.7).

Witness and coefficients. Every declaration and every arrow-defined value is committed, including values with purely linear participation such as row sums, with one stated exception: the chunked declaration of B.1 commits a value's words while the whole value remains notation. Constraint coefficients may be arbitrary public functions of the challenges (the product coefficients $\lambda[a]\ \rho[b]$ in the matmul pin; the per-entry coefficients $\alpha - v[j]$ in the LogUp settlements), derived identically by both sides and never materialized. A value that appears in no line, such as the per-cell broadcast product inside RMSNorm's projections (B.4), is never committed at all; the claim reaches it only through projections.

Turn boundaries. Each horizontal rule in a listing is a message boundary: the lines between two rules are one party's message, committed together, and a `chal` line is drawn only after everything above its rule is committed. This ordering is what each lemma's "committed before the challenge" step refers to. The composite lines (`range`, `lookup`, `rescale`) span turns without showing it: their visible content commits at the line's position, and their challenge-dependent companions (folded keys, inverses) commit after the table's challenges, per B.1. Hiding the split is safe because the visible half is the half whose timing matters: LogUp soundness needs the queries' values and the multiplicity histograms fixed before the table's challenges are drawn, which is exactly what the line's pre-challenge position shows, while the deferred companions are functions of the challenges, their timing forced, with no prover choice in when they land. The block above the first rule (`input` lines) is inherited context, not part of the claim's transcript, and each claim's guarantees are conditional on its inputs, the weights included.

A lemma's width or exclusion argument may, however, rely on a magnitude bound established by an upstream claim, as B.6's gap exclusion relies on the router logits being bounded by their producing matmul's rescale. Such reliance is stated in the lemma with the upstream source named; these cross-claim width dependencies are part of what the claim-graph audit of §7.2 must trace.

What may be factored out. The reader should be able to trust that nothing hidden matters, so the rule is strict: exactly three line kinds carry an implied expansion, `range`, `lookup`, and `rescale`, and an expansion is permitted only because it is soundness-inert, meaning no lemma in this appendix ever needs its internals beyond the entry in B.1. Under this rule the LogUp bookkeeping is factored everywhere: the per-table challenges, folded keys, inverses, and settlements are identical boilerplate whose soundness is the per-table LogUp term of §7.3's error sum, cited by no claim lemma, and each `lookup` or `range` line implies them (B.1). Anything a lemma touches stays inline: the softmax saturation mechanics, the bracket slacks, and every pin appear in their listings explicitly.

Counting. The last three columns of every listing carry each line's contribution to the three drivers (witness slots W, linear constraints L, quadratic products Q), written in the claim's size parameters, with a dot for zero; an italic totals row closes each listing, so verifying a claim's counts is summing columns. Two principles fix the cells.

Constraints: a constraint line (arrow, `==`, `range`, `lookup`, `rescale`) contributes one constraint per assignment of its free indices, once per comma-separated clause, respecting any filter on the extent. Indices bound by a summation inside the equation contribute nothing to the multiplicity. So the vector equality $\boldsymbol{s}_1[h,q] + \boldsymbol{r}_{\mathrm{lo}}[h,q] == \mathrm{s_y}$, free in h and q, is one constraint per row, while the matmul pin, whose every index is summed, is a single constraint however many committed values it touches. The tag names the test the constraints feed; a `range` contributes its quadratic, the signed form also one linear (the shift relation), and a `lookup` one linear (the key folding) and one quadratic (the inverse).

Witness: each variable a line introduces contributes one slot per element of its extent. Declared variables and arrow left-sides are introduced visibly; a `range` also introduces its lookup inverse (the signed form additionally the shifted copy), and a `lookup` its value, folded key, and inverse; a `rescale` its kept word, low word, shifted copy, and two inverses. `input` and `chal` lines introduce nothing committed.

For reference, the footprints per element these principles give:

line kind	W	L	Q
`input`, `chal`	0	0	0
`decl`	1 per variable	0	0
`decl` (chunked, t words)	$2t$	0	t
`lin` arrow	1	1	0
`quad` arrow	1	0	1
`lin ==`	0	1	0

line kind	W	L	Q
`quad ==`	0	0	1
`range` $\sqsubseteq$	1	0	1
`range` $\sqsubseteq \pm$	2	1	1
`lookup`	3	1	1
`rescale`	5	2	2

A count cell is its line's footprint times its extent. A filtered extent counts its filtered size, as in the softmax triangle; B.4's carry-chain lines state both chains at once, and their cells include the factor of two. How many committed values an equation touches is not tracked: the total across all constraints is proportional to W (each slot appears in a handful of constraints, the 2 to $4 \times W$ of Appendix C) and is priced through W in the cost model. LogUp settlements, multiplicity histograms, and challenges are per registry table, shared across all claims, and appear in the registry rather than in any claim's counts.

Every declaration is argued. Each claim ends with a soundness lemma, and its job is to show that the prover had no unaccounted freedom in the witness. Values introduced by arrow lines need no argument: by construction, their constraints admit exactly one satisfying assignment once everything above them is fixed. The work concerns the declared values, the ones the prover asserts freely. For each declaration, the lemma shows one of three things: that the constraints pin it to a single possible value; or, where a pin is deliberately one-sided, that every remaining choice moves the reported result in the safe direction; or, where a declaration is left unconstrained in some region (a flag's inverse where the flag is zero, a masked cell's key), that the choice is value-neutral, meaning no downstream committed value or public output depends on it. Declarations may be argued in any order that avoids circularity, and a `range`, `lookup`, or `rescale` line inherits its B.1 lemma at the invoked size. Each lemma also classifies how the claim behaves under overflow, following the cases of §3.6. A declaration without an argued pin is a defect by definition.

B.1 Shared machinery

Three composites recur across the claims as their own line kinds, the range check, the paired lookup, and the rescale, and one shared pattern, the word decomposition, recurs written out inline. The LogUp lookup is the mechanism underlying the first two, and the decomposition and rescale build on the range check, so each entry below depends only on entries above it. All of them query public tables, and every table lives in one registry, given first.

B.1.0 Table registry

Each row records one public table: its contents, length, which claims query it, the query volume $M(S)$ summed over the Maverick claim list as a polynomial in the context length S (48 layers, $n_q = 40$, $E = 128$, $V = 202{,}048$; §A.2), and the per-table LogUp soundness term $(M + T_{\text{len}} + 1)/|F|$, with T_{len} the table length, evaluated in the error column at $|F| \approx 2^{64}$ and the demonstrated $S = 1000$. In the sharers column, N, B, T, E are the invoking claim's size parameters.

table	contents	length	queried by (per instance)	$M(S)$	error at S=1000
range_2	identity on $[0, 2^2)$	4	SiLU word $\boldsymbol{a}_0$ (N)	786,432 S	$2^{-34.4}$
range_5	identity on $[0, 2^5)$	2^5	RMSNorm $\boldsymbol{y}-1$ top chunk (B)	97 S	$2^{-47.4}$
range_9	identity on $[0, 2^9)$	2^9	RMSNorm $\boldsymbol{G}_2$ top chunks ($2B$)	194 S	$2^{-46.4}$
range_{10}	identity on $[0, 2^{10})$	2^{10}	RMSNorm $\boldsymbol{g}_{0h}$ top chunks ($2B$)	194 S	$2^{-46.4}$
range_{11}	identity on $[0, 2^{11})$	2^{11}	routing gap words ($3TE$); hidden-token select gap words ($2TV$, B.8); RMSNorm $\boldsymbol{g}_{1h}$ and slack top chunks ($4B$)	413,700 S	$2^{-35.4}$

table	contents	length	queried by (per instance)	$M(S)$	error at S=1000
range_{12}	identity on $[0, 2^{12})$	2^{12}	rescale low words (all ⓡ claims); surprisal slack words (4 per position)	$1{,}920\ S^2 + 71{,}190{,}852\ S$	$2^{-27.9}$
range_{14}	identity on $[0, 2^{14})$	2^{14}	SiLU word $\boldsymbol{a}_4$ (N)	$786{,}432\ S$	$2^{-34.4}$
range_{16}	identity on $[0, 2^{16})$	2^{16}	softmax $\boldsymbol{z}_{\text{high}}$ ($n_q S^2$); SiLU $\boldsymbol{a}_2, \boldsymbol{a}_3$ ($2N$); RMSNorm 16-bit chunks ($17B$); surprisal remainder (1 per position)	$1{,}920\ S^2 + 1{,}574{,}514\ S$	$2^{-32.3}$
range_{18}	identity on $[0, 2^{18})$	2^{18}	RMSNorm $\boldsymbol{S}_{\text{tot}}$ limbs and carry lows ($7B$)	$679\ S$	$2^{-44.2}$
range_{20}	identity on $[0, 2^{20})$	2^{20}	surprisal argmax gaps (V per position, alongside Exp's key bound)	$202{,}048\ S$	$2^{-36.4}$
range_{24}	identity on $[0, 2^{24})$	2^{24}	softmax slacks ($2n_q S$) and the shift's signed copy ($n_q S$)	$5{,}760\ S$	$2^{-39.6}$
range_{26}	identity on $[0, 2^{26})$	2^{26}	rescale shifted words (all ⓡ claims)	$1{,}920\ S^2 + 71{,}190{,}848\ S$	$2^{-27.9}$
$\mathrm{T_A}, \mathrm{T_B}$	$\mathrm{round}(\mathrm{s_y}\, e^{-j/\mathrm{s_c}})$, δ-shifted pair, half-to-even; doubled with a zero half, targeted by the masked-key term $[[i > q]]$; zero from index 36{,}909 / 36{,}910	$2\ \mathrm{Z_{max}} = 80{,}000$	softmax, every attention layer ($n_q S^2$ per table)	$1{,}920\ S^2$ each	$2^{-33.2}$
silu	branch-concatenated, bin width 4, bin-centre rounding	2^{15}	SiLU (N)	$786{,}432\ S$	$2^{-34.4}$
sigmoid	$\mathrm{round}(\sigma((j - 2^{18})/\mathrm{S}) \cdot \mathrm{S})$	2^{19}	routing weight (T per MoE layer)	$24\ S$	$2^{-44.9}$
Exp	$\max(1, \lceil \mathrm{s_y}\, e^{-g^2/\mathrm{s_c}} \rceil)$	2^{20}	surprisal (V per position)	$202{,}048\ S$	$2^{-36.4}$
Pow	$\lfloor \mathrm{s_y}\, e^{j/\mathrm{s_b}} \rfloor$	$\lceil \mathrm{s_b} \ln V \rceil + 4 = 50{,}042$	surprisal (1 per position)	S	$2^{-48.4}$

Each row also carries its own constants: for the attention pair, $\delta = 1$, $\mathrm{Z}_{\max} = 40{,}000$, and $\mathrm{s_y} = \mathrm{s_c} = \mathrm{S}$; the surprisal rows use $\mathrm{s_c} = \mathrm{s_y} = 2^{28}$ and $\mathrm{s_b} = 2^{12}$; the sigmoid row's shift is 2^{18}. Token-binding tables join the registry when Appendix E's binding runs. One resident multiplicity histogram per row is the persistent prover state of §6.3.

The two rescale rows, at $2^{-27.9}$, are the binding terms of §7.3's LogUp sum, quoted there and checkable here. The S^2 terms are attention's: the exponential pair, the saturation words, and the scores matmul's share of the two rescale rows; every other row grows linearly with context. Rows are keyed by table contents. The implementation registers a few same-content families as separate LogUp instances (each routing invocation's word table; the three width-16 families); this splits a row's M and histogram across copies and adds only the extra settlements' $(T_{\text{len}} + 1)/|F|$ terms, negligible against every row above.

LogUp lookups. Two of the composites below, the range check and the paired lookup, are lookup arguments, built on LogUp as described in §3.4. A query against a public table is certified by committing, after the table's challenges (α, β) are drawn, a folded key that combines the query's components under β and the inverse $\boldsymbol{z} = 1/(\alpha - \text{key})$; a per-table settlement then proves that the multiset of all queries matches the table. The settlement commits a multiplicity histogram (before the challenges) and per-entry weights $\boldsymbol{w}[j] = \boldsymbol{m}[j]/(\alpha - v[j])$ (after), and emits one linear constraint per entry, with the public coefficient $\alpha - v[j]$, plus a single cross-claim sum identity equating the query inverses with the entry weights. Because the challenge folds the query's components together, a query matches an entry only when every component agrees. Shared by all lookups, and per the factoring rule implied rather than listed: the challenge turns, the folded keys and inverses, the settlement, and the histogram. The challenges are sampled per table from the round seed.

Range check. A range check proves that a committed value $\boldsymbol{x}$ lies in $[0, 2^w)$: a LogUp lookup against the table containing every integer in that range, keyed on $\boldsymbol{x}$ alone, so no folded key is needed and the per-query constraint is the single quadratic $(\alpha - \boldsymbol{x})\ \boldsymbol{z} = 1$. In listings it is the `range` line. Counts per checked slot: one witness slot ($\boldsymbol{z}$) and one quadratic.

A range check as stated rejects negative values, whose field representatives lie near P rather than in $[0, 2^w)$. To check a signed value $\boldsymbol{x} \in [-2^{w-1}, 2^{w-1})$, which is the `range` line's $\pm$ form, the prover commits the shifted copy $\boldsymbol{x}_{\text{shifted}} = \boldsymbol{x} + 2^{w-1}$, one linear constraint ties it to $\boldsymbol{x}$, and the range check runs on $\boldsymbol{x}_{\text{shifted}}$; the signed range follows from the offset. The signed form adds one slot and one linear constraint.

Paired lookup. A paired lookup proves a functional relation $\boldsymbol{y} = \mathrm{T}[\boldsymbol{x}]$ against a table of input-output pairs; it is the expansion of the `lookup` line. The LogUp query is the pair, folded as $\boldsymbol{u} = (\boldsymbol{x} + \text{shift}) + \beta\ \boldsymbol{y}$, so one lookup certifies both that $\boldsymbol{x}$ is in the table's key range and that $\boldsymbol{y}$ is the table's value there. Committed per query: the value $\boldsymbol{y}$, the folded key $\boldsymbol{u}$, and the inverse $\boldsymbol{z}$; the input $\boldsymbol{x}$ belongs to its producer. Per-query constraints: one linear (the key folding) and one quadratic (the inverse). Counts per query: three slots, one linear, one quadratic, matching A.1's paired-lookup row.

Word decomposition. A word decomposition proves that a wide value is built from narrow pieces in a fixed way: the single linear constraint

$$\boldsymbol{x} = \textstyle\sum_n \text{coeff}_n \cdot \boldsymbol{word}_n,$$

with each word separately range-checked (signed words via the shifted form above). The coefficients are public constants of the invoking claim, powers of two chosen so the words' windows tile the intended range, as in splitting a product into a kept high word and a dropped low word. A decomposition is not a line kind: listings write it out, declaring the words, range-checking each into its width's registry table, and tying them to $\boldsymbol{x}$ with one linear `==`, and the lemma cites Lemma B.1a for the pattern's uniqueness. The softmax key split (B.3) is the worked example.

The point of the pattern is uniqueness, and it is worth seeing why it holds. The linear constraint fixes the weighted sum; each range check confines its word to a window; and because the windows tile, no two distinct word assignments produce the same sum. So exactly one assignment satisfies all the constraints, with one proviso. The argument runs over the integers, but the constraints run over the field, and a wrapped negative value has a representative near P. Uniqueness therefore additionally requires that the largest value the words can recompose, $\sum_n \text{coeff}_n(2^{w_n} - 1)$ plus any shift, lies below P: then no wrapped value is reachable by any valid assignment. This is the width condition, and every claim that leans on a decomposition for an exclusion argument must state it with its actual widths. Overflow of a decomposed operand is thus the rejecting case of §3.6: excluded by construction rather than priced by the bound.

Counts for a t-word decomposition: t word slots plus their range checks, one linear constraint plus any signed shifts, t quadratics.

One variant serves values that are only ever used through their pieces: a **chunked declaration**, written `decl` $\boldsymbol{g}$ (chunks $w_1, \ldots, w_t$), commits only the words. The whole value $\boldsymbol{g}$ is never committed; the bare symbol abbre-

viates the weighted chunk sum wherever later lines use it. Footprint: $2t$ slots (words and inverses) and t quadratics, with no linear constraint, since the sum is notation rather than a constraint. The carry chains of B.4 are its users.

Soundness (Lemma B.1a). As argued above: given the linear constraint, the range checks, and the width condition, exactly one word assignment satisfies the constraints.

Rescale. A rescale returns a raw product to the working scale. A product of two values at scale S arrives at scale S^2; the rescale keeps the signed high word $\boldsymbol{x}$ and drops the $r = \log_2(s_a s_b / s_{\text{out}})$ low bits, which are the quantization error the bound prices. It is the two-word instance of the decomposition, in the form the compiler emits: the decomposition constraint $\boldsymbol{x}_{\text{full}} - 2^r \boldsymbol{x} - \boldsymbol{x}_{\text{low}} = 0$, the signed-shift relation $\boldsymbol{x}_{\text{shifted}} - \boldsymbol{x} = 2^{w-1}$, and the two range-check quadratics on $\boldsymbol{x}_{\text{low}}$ and $\boldsymbol{x}_{\text{shifted}}$. In listings it is the `rescale` line, $\boldsymbol{x} \leftarrow$ `rescale`$(\boldsymbol{x}_{\text{full}})$; the low word queries range_{12} and the shifted word range_{26} (registry).

Counts per element: five witness slots ($\boldsymbol{x}$, $\boldsymbol{x}_{\text{low}}$, $\boldsymbol{x}_{\text{shifted}}$, two inverses), two linear, two quadratic; the input $\boldsymbol{x}_{\text{full}}$ is counted by the invoking claim. The demonstrated widths are $r = 12$ and $w = 26$, so the kept word satisfies $\boldsymbol{x} \in [-2^{25}, 2^{25})$.

Soundness (Lemma B.1b). Lemma B.1a at two words with the signed form. The width condition at the demonstrated parameters: the maximum recomposable value is $2^{12} \cdot 2^{25} + (2^{12} - 1) \approx 2^{37}$, against $P \approx 2^{64}$, about 27 bits of margin, so no wrapped negative has a valid decomposition and the kept word is the unique high part of the committed product.

B.2 Matmul

Matmul proves $\boldsymbol{C}_{\text{full}} = \boldsymbol{A}\boldsymbol{B}$ for $\boldsymbol{A} \in F^{m \times k}$ and $\boldsymbol{B} \in F^{k \times n}$; the output consumed downstream is the rescale of $\boldsymbol{C}_{\text{full}}$ to the working scale, committed in the same pre-challenge message.

			W	L	Q
`input`	$\boldsymbol{A}$	$\forall\, a \in [m],\ j \in [k]$	$\cdot$	$\cdot$	$\cdot$
`input`	$\boldsymbol{B}$	$\forall\, j,\ b \in [n]$	$\cdot$	$\cdot$	$\cdot$
	raw product at $s_a s_b$				
`decl`	$\boldsymbol{C}_{\text{full}}$	$\forall\, a, b$	mn	$\cdot$	$\cdot$
`rescale`	$\boldsymbol{C}[a,b] \leftarrow$ `rescale`$(\boldsymbol{C}_{\text{full}}[a,b])$	$\forall\, a, b$	$5mn$	$2mn$	$2mn$
`chal`	ρ	$\forall\, b$	$\cdot$	$\cdot$	$\cdot$
`chal`	λ	$\forall\, a$	$\cdot$	$\cdot$	$\cdot$
`lin`	$\boldsymbol{y}[j] \leftarrow \sum_{b \in [n]} \boldsymbol{B}[j,b]\ \rho[b]$	$\forall\, j$	k	k	$\cdot$
`lin`	$\boldsymbol{u}[j] \leftarrow \sum_{a \in [m]} \lambda[a]\ \boldsymbol{A}[a,j]$	$\forall\, j$	k	k	$\cdot$
`quad`	$\boldsymbol{p}[j] \leftarrow \boldsymbol{u}[j]\ \boldsymbol{y}[j]$	$\forall\, j$	k	$\cdot$	k
`lin`	$\sum_{a,b} \lambda[a]\ \rho[b]\ \boldsymbol{C}_{\text{full}}[a,b] == \sum_j \boldsymbol{p}[j]$		$\cdot$	1	$\cdot$
totals			$6mn + 3k$	$2mn + 2k + 1$	$2mn + k$

The totals row matches A.1's matmul row at $H = 1$.

Soundness (Lemma B.2). The one declaration is $\boldsymbol{C}_{\text{full}}$. The arrow lines define $\boldsymbol{y}$, $\boldsymbol{u}$, $\boldsymbol{p}$ exactly: given the commitments and challenges, each has one satisfying value, and by associativity $\sum_j \boldsymbol{p}[j] = (\lambda^\top \boldsymbol{A})(\boldsymbol{B}\rho) = \lambda^\top(\boldsymbol{A}\boldsymbol{B})\rho$. The pin therefore enforces $\lambda^\top \boldsymbol{C}_{\text{full}}\ \rho = \lambda^\top(\boldsymbol{A}\boldsymbol{B})\rho$, i.e. $\lambda^\top E \rho = 0$ for $E = \boldsymbol{C}_{\text{full}} - \boldsymbol{A}\boldsymbol{B}$. If $E \neq 0$, then $E\rho \neq 0$ except with probability $1/|F|$ over ρ: a nonzero matrix has a nonzero row, and that row's inner product with a uniform ρ is uniform. Conditioned on $E\rho \neq 0$, $\lambda^\top(E\rho) = 0$ with probability $1/|F|$ over λ. A false $\boldsymbol{C}_{\text{full}}$ survives with probability at most $2/|F|$, and since it is committed before (ρ, λ) are drawn, the prover cannot select it against the challenge; each matmul adds $2/|F|$ to the error sum of §7.3. Given $\boldsymbol{C}_{\text{full}}$, the output $\boldsymbol{C}$ is unique by Lemma B.1b. Overflow in the projection constraints is the accepting case of §3.6: field identities, wrapped values unique but wrong, priced by the bound; the magnitude exclusion lives in the rescale (Lemma B.1b).

Generalization. H-head batches share one challenge pair, with one pin constraint per head (coefficients $\lambda[a]\ \rho[b]$ on $\boldsymbol{C}_{\text{full}}[a,h,b]$), the $O(k)$ terms counted once across heads: A.1's $+H$ term. `transpose_b` reindexes $\boldsymbol{B}$ only. Attention-shaped instances are written in B.3's (h,q,i) indexing: the scores matmul's output extent is $\forall\, h \in [n_q],\ q \in [S],\ i \in [S]$ with one pin per head h, so its cells align with softmax's and with A.3's per-cell accounting.

B.3 Softmax

Softmax proves, per attention head and query position, the exponentiated causal scores at scale $\mathrm{s_y}$, normalized by pinning the per-row shift. The tables $\mathrm{T_A}$ and $\mathrm{T_B}$, with their rounding rule and constants, are specified in the registry (B.1.0).

input	$\boldsymbol{x}$	$\forall\, h \in [n_q],\ q \in [S],\ i \in [S]$	W ·	L ·	Q ·
	— shift and exponentiate —				
	per-row shift				
decl	$\boldsymbol{c}$	$\forall\, h, q$	$n_q S$	·	·
range	$\boldsymbol{c}[h,q] \sqsubseteq \pm\mathrm{range}_{24}$	$\forall\, h, q$	$2n_q S$	$n_q S$	$n_q S$
	saturation words				
decl	$\boldsymbol{z}_{\text{high}}$	$\forall\, h, q, i$	$n_q S^2$	·	·
range	$\boldsymbol{z}_{\text{high}}[h,q,i] \sqsubseteq \mathrm{range}_{16}$	$\forall\, h, q, i$	$n_q S^2$	·	$n_q S^2$
lin	$\boldsymbol{z}[h,q,i] \leftarrow \boldsymbol{c}[h,q] - \boldsymbol{x}[h,q,i] - \mathrm{Z}_{\max}\, \boldsymbol{z}_{\text{high}}[h,q,i]$	$\forall\, h, q,\ i \le q$	$\frac{1}{2} n_q S(S{+}1)$	$\frac{1}{2} n_q S(S{+}1)$	·
	free in key range; value-neutral				
decl	$\boldsymbol{z}[h,q,i]$	$\forall\, h, q,\ i > q$	$\frac{1}{2} n_q S(S{-}1)$	·	·
lookup	$\boldsymbol{e}_1[h,q,i] \leftarrow \mathrm{T_A}\big[\, \boldsymbol{z}[h,q,i] + \mathrm{Z}_{\max} \cdot [\![\, i > q \,]\!] \,\big]$	$\forall\, h, q, i$	$3n_q S^2$	$n_q S^2$	$n_q S^2$
lookup	$\boldsymbol{e}_2[h,q,i] \leftarrow \mathrm{T_B}\big[\, \boldsymbol{z}[h,q,i] + \mathrm{Z}_{\max} \cdot [\![\, i > q \,]\!] \,\big]$	$\forall\, h, q, i$	$3n_q S^2$	$n_q S^2$	$n_q S^2$
	— saturate the tail —				
	free at $\boldsymbol{z}_{\text{high}} = 0$; value-neutral				
decl	$\boldsymbol{inv}$	$\forall\, h, q, i$	$n_q S^2$	·	·
quad	$\boldsymbol{t}[h,q,i] \leftarrow \boldsymbol{z}_{\text{high}}[h,q,i] \cdot \boldsymbol{inv}[h,q,i]$	$\forall\, h, q, i$	$n_q S^2$	·	$n_q S^2$
quad	$\boldsymbol{t}[h,q,i] \cdot \boldsymbol{z}_{\text{high}}[h,q,i] == \boldsymbol{z}_{\text{high}}[h,q,i]$	$\forall\, h, q, i$	·	·	$n_q S^2$
quad	$\boldsymbol{t}[h,q,i]^2 == \boldsymbol{t}[h,q,i]$	$\forall\, h, q, i$	·	·	$n_q S^2$
quad	$\boldsymbol{mux}_1[h,q,i] \leftarrow \boldsymbol{t}[h,q,i] \cdot \boldsymbol{e}_1[h,q,i]$	$\forall\, h, q, i$	$n_q S^2$	·	$n_q S^2$
quad	$\boldsymbol{mux}_2[h,q,i] \leftarrow \boldsymbol{t}[h,q,i] \cdot \boldsymbol{e}_2[h,q,i]$	$\forall\, h, q, i$	$n_q S^2$	·	$n_q S^2$
lin	$\boldsymbol{y}_1[h,q,i] \leftarrow \boldsymbol{e}_1[h,q,i] - \boldsymbol{mux}_1[h,q,i]$	$\forall\, h, q, i$	$n_q S^2$	$n_q S^2$	·
lin	$\boldsymbol{y}_2[h,q,i] \leftarrow \boldsymbol{e}_2[h,q,i] - \boldsymbol{mux}_2[h,q,i]$	$\forall\, h, q, i$	$n_q S^2$	$n_q S^2$	·
	— bracket the shift —				
lin	$\boldsymbol{s}_1[h,q] \leftarrow \sum_{i \in [S]} \boldsymbol{y}_1[h,q,i]$	$\forall\, h, q$	$n_q S$	$n_q S$	·
lin	$\boldsymbol{s}_2[h,q] \leftarrow \sum_{i \in [S]} \boldsymbol{y}_2[h,q,i]$	$\forall\, h, q$	$n_q S$	$n_q S$	·
	bracket slacks				
decl	$\boldsymbol{r}_{\text{lo}},\ \boldsymbol{r}_{\text{hi}}$	$\forall\, h, q$	$2n_q S$	·	·
range	$\boldsymbol{r}_{\text{lo}}[h,q] \sqsubseteq \mathrm{range}_{24}, \quad \boldsymbol{r}_{\text{hi}}[h,q] \sqsubseteq \mathrm{range}_{24}$	$\forall\, h, q$	$2n_q S$	·	$2n_q S$
lin	$\boldsymbol{s}_1[h,q] + \boldsymbol{r}_{\text{lo}}[h,q] == \mathrm{s_y}$	$\forall\, h, q$	·	$n_q S$	·
lin	$\boldsymbol{r}_{\text{hi}}[h,q] - \boldsymbol{s}_2[h,q] == -(\mathrm{s_y} + 1)$	$\forall\, h, q$	·	$n_q S$	·
totals			$15\, n_q S^2 + 9\, n_q S$	$\frac{1}{2} n_q S(S{+}1) + 4\, n_q S^2 + 5\, n_q S$	$8\, n_q S^2 + 3\, n_q S$

The totals row matches the emitted counts and A.1's row at $B = n_q S$, $M = S$. The half-terms are the causal filter: the definition line and the complement declaration split each head's S^2 cells into $\frac{1}{2}S(S{+}1)$ unmasked and $\frac{1}{2}S(S{-}1)$ masked.

Soundness (Lemma B.3). The declarations are $\boldsymbol{c}$, $\boldsymbol{z}_{\text{high}}$, the masked $\boldsymbol{z}$, $\boldsymbol{inv}$, $\boldsymbol{r}_{\text{lo}}$, $\boldsymbol{r}_{\text{hi}}$. $\boldsymbol{z}_{\text{high}}$: given $\boldsymbol{c}$, the pair $(\boldsymbol{z}, \boldsymbol{z}_{\text{high}})$ is unique on unmasked cells by Lemma B.1a, $\boldsymbol{z}$'s window from the lookup key range and $\boldsymbol{z}_{\text{high}}$'s from its range pin; the width condition spans $\mathrm{Z}_{\max} \cdot 2^{16} \approx 2^{31.3} \ll P$. The masked $\boldsymbol{z}$: for $i > q$ the lookup key lies in the zero half, so $\boldsymbol{e}_1 = \boldsymbol{e}_2 = 0$ whatever value is committed; the freedom is value-neutral, and by the same reading token i contributes nothing to row q's sums, outputs, or shift. Causality then holds globally by induction: every attention claim carries the filter $i \le q$ in its key, and every other claim in the graph is position-local, so position q's logits depend only on tokens $\le q$ (the position-locality of the non-attention claims is a claim-graph fact the audit of §7.2 confirms). $\boldsymbol{inv}$: at $\boldsymbol{z}_{\text{high}} \neq 0$ the flag constraints force $\boldsymbol{t} = 1$, $\boldsymbol{inv} = 1/\boldsymbol{z}_{\text{high}}$, unique; at $\boldsymbol{z}_{\text{high}} = 0$ they force $\boldsymbol{t} = 0$ and leave $\boldsymbol{inv}$ free, value-neutral. The slacks: fixed by their equalities once $\boldsymbol{s}_1, \boldsymbol{s}_2$ are, non-negative by their range pins, so the pins state $\boldsymbol{s}_1(\boldsymbol{c}) \le \mathrm{s_y}$ and $\boldsymbol{s}_2(\boldsymbol{c}) \ge \mathrm{s_y} + 1$. For $\boldsymbol{c}$: the lookups bound every unmasked key, so $\boldsymbol{c} \ge \max_{i \le q} \boldsymbol{x}[h,q,i]$; the tables are bit-identical up to δ and reach zero before $\mathrm{Z}_{\max}$ (registry), so a saturated cell equals what an unbounded table would return and $\boldsymbol{s}_2(\boldsymbol{c}) = \boldsymbol{s}_1(\boldsymbol{c} - \delta)$ over the muxed sums as exact integers; $\boldsymbol{s}_1$ is monotone non-increasing in $\boldsymbol{c}$; hence exactly one integer $\boldsymbol{c}$ satisfies both pins. Overflow: the rejecting case of §3.6 at the bracket, honest-fit $S\, \mathrm{s_y} \lesssim 2^{24}$ limiting $S \lesssim 4096$ at $\mathrm{s_y} = 2^{12}$, all checked values below $2^{24} \ll P$; elsewhere the accepting case.

Scope. The claim supports index-predicate masks whose predicate implies $i \le q$, compiled in as public structure; a sliding window, $q - \mathrm{w} < i \le q$, is the worked example, and preserves the causality reading since the upper edge is unchanged. Arbitrary mask inputs are out of scope: admitting one would replace the syntactic causality check with a per-deployment audit of mask support, reopening the obligation the fixed predicate discharges. The non-causal form is the empty predicate with an undoubled table. A counting remark: the definition line's extent is $\sum_q (q{+}1) = S(S{+}1)/2$ per head, the diagonal $i = q$ included, which is A.1's $+1$.

B.4 RMSNorm

RMSNorm proves $\boldsymbol{out}[b,i] = \boldsymbol{x}[b,i]\ \boldsymbol{y}[b]$ with $\boldsymbol{y}[b]$ the rounded reciprocal square root $\lceil \sqrt{\mathrm{magic}/\boldsymbol{S}_{\text{tot}}[b]} \rceil$, where $\boldsymbol{S}_{\text{tot}}[b] = \sum_i \boldsymbol{x}[b,i]^2 + d\,\varepsilon$ and $\mathrm{magic} = d\,\mathrm{S}^4$, over B rows of width d. It is the one nonlinearity with no lookup table: the rsqrt is pinned by an algebraic bracket, two inequalities that only the correct rounding satisfies. An inequality means

nothing over a field, so the bracket must compare genuine integers: the products $\boldsymbol{y}^2\,\boldsymbol{S}_{\text{tot}}$ are too wide to commit whole, and each is instead assembled limb by limb through a carry chain whose every part is range-checked into a window no step can wrap. All windows are derived from $(d, \mathrm{S}, \varepsilon)$ by both sides independently, never chosen by the prover. At the demonstrated parameters the limb width is 18; the $\boldsymbol{y}$ window is 21 bits, split $(16, 5)$; the slack window 59 bits, split $(16, 16, 16, 11)$; and the carry windows 42, 43, and 25 bits, chunked $(16, 16, 10)$, $(16, 16, 11)$, $(16, 9)$.

			W	L	Q
`input`	$\boldsymbol{x}$	$\forall\, b \in [B],\ i \in [d]$	·	·	·
	— row energy —				
`quad`	$\boldsymbol{X}_{\text{sq}}[b,i] \leftarrow \boldsymbol{x}[b,i]^2$	$\forall\, b, i$	Bd	·	Bd
`lin`	$\boldsymbol{S}_{\text{sum}}[b] \leftarrow \sum_{i\in[d]} \boldsymbol{X}_{\text{sq}}[b,i]$	$\forall\, b$	B	B	·
`lin`	$\boldsymbol{S}_{\text{tot}}[b] \leftarrow \boldsymbol{S}_{\text{sum}}[b] + d\,\varepsilon$	$\forall\, b$	B	B	·
	— the rsqrt and its windows —				
	the rsqrt scalars				
`decl`	$\boldsymbol{y}$	$\forall\, b$	B	·	·
`lin`	$\boldsymbol{y}_{m1}[b] \leftarrow \boldsymbol{y}[b] - 1$	$\forall\, b$	B	B	·
	window words of $\boldsymbol{y} - 1$				
`decl`	$\boldsymbol{y}_{w0},\ \boldsymbol{y}_{w1}$	$\forall\, b$	$2B$	·	·
`range`	$\boldsymbol{y}_{w0}[b] \sqsubseteq \text{range}_{16}, \quad \boldsymbol{y}_{w1}[b] \sqsubseteq \text{range}_{5}$	$\forall\, b$	$2B$	·	$2B$
`lin`	$\boldsymbol{y}_{m1}[b] == \boldsymbol{y}_{w0}[b] + 2^{16}\boldsymbol{y}_{w1}[b]$	$\forall\, b$	·	B	·
	limbs of $\boldsymbol{S}_{\text{tot}}$				
`decl`	$\boldsymbol{S}_0,\ \boldsymbol{S}_1,\ \boldsymbol{S}_2$	$\forall\, b$	$3B$	·	·
`range`	$\boldsymbol{S}_\ell[b] \sqsubseteq \text{range}_{18}$	$\forall\, b,\ \ell \in \{0,1,2\}$	$3B$	·	$3B$
`lin`	$\boldsymbol{S}_{\text{tot}}[b] == \boldsymbol{S}_0[b] + 2^{18}\boldsymbol{S}_1[b] + 2^{36}\boldsymbol{S}_2[b]$	$\forall\, b$	·	B	·
`quad`	$\boldsymbol{q}_1[b] \leftarrow \boldsymbol{y}[b]^2$	$\forall\, b$	B	·	B
`quad`	$\boldsymbol{q}_2[b] \leftarrow \boldsymbol{y}_{m1}[b]^2$	$\forall\, b$	B	·	B
`quad`	$\boldsymbol{H}_\ell[b] \leftarrow \boldsymbol{q}[b]\ \boldsymbol{S}_\ell[b]$	$\forall\, b,\ \ell;$ per chain $\boldsymbol{q} \in \{\boldsymbol{q}_1, \boldsymbol{q}_2\}$	$6B$	·	$6B$
	— carry chain, per chain —				
	carry high parts				
`decl`	$\boldsymbol{g}_{0h}$ (chunks $16, 16, 10$), $\boldsymbol{g}_{1h}$ (chunks $16, 16, 11$)	$\forall\, b$	$24B$	·	$12B$
`lin`	$\boldsymbol{g}_{0l}[b] \leftarrow \boldsymbol{H}_0[b] - 2^{18}\,\boldsymbol{g}_{0h}[b]$	$\forall\, b$	$2B$	$2B$	·
`lin`	$\boldsymbol{g}_{1l}[b] \leftarrow \boldsymbol{H}_1[b] + \boldsymbol{g}_{0h}[b] - 2^{18}\,\boldsymbol{g}_{1h}[b]$	$\forall\, b$	$2B$	$2B$	·
`range`	$\boldsymbol{g}_{0l}[b] \sqsubseteq \text{range}_{18}, \quad \boldsymbol{g}_{1l}[b] \sqsubseteq \text{range}_{18}$	$\forall\, b$	$4B$	·	$4B$
	top accumulator				
`decl`	$\boldsymbol{G}_2$ (chunks $16, 9$)	$\forall\, b$	$8B$	·	$4B$
`lin`	$\boldsymbol{H}_2[b] + \boldsymbol{g}_{1h}[b] == \boldsymbol{G}_2[b]$	$\forall\, b$	·	$2B$	·
	— bracket pins —				
	bracket slacks				
`decl`	$\boldsymbol{s}_{\text{lo}},\ \boldsymbol{s}_{\text{hi}}$	$\forall\, b$	$2B$	·	·
	slack words				
`decl`	$\boldsymbol{w}_{\text{lo}},\ \boldsymbol{w}_{\text{hi}}$	$\forall\, b,\ n \in [4]$	$8B$	·	·
`range`	$\boldsymbol{w}_{\text{lo}}[b,n] \sqsubseteq \text{range}_{16}, \quad \boldsymbol{w}_{\text{hi}}[b,n] \sqsubseteq \text{range}_{16}$	$\forall\, b,\ n \le 2$	$6B$	·	$6B$
`range`	$\boldsymbol{w}_{\text{lo}}[b,3] \sqsubseteq \text{range}_{11}, \quad \boldsymbol{w}_{\text{hi}}[b,3] \sqsubseteq \text{range}_{11}$	$\forall\, b$	$2B$	·	$2B$
`lin`	$\boldsymbol{s}_{\text{lo}}[b] == \sum_{n\in[4]} 2^{16n}\ \boldsymbol{w}_{\text{lo}}[b,n]$	$\forall\, b$	·	B	·
`lin`	$\boldsymbol{s}_{\text{hi}}[b] == \sum_{n\in[4]} 2^{16n}\ \boldsymbol{w}_{\text{hi}}[b,n]$	$\forall\, b$	·	B	·
`lin`	$2^{36}\boldsymbol{G}_2[b] + 2^{18}\boldsymbol{g}_{1l}[b] + \boldsymbol{g}_{0l}[b] - \boldsymbol{s}_{\text{lo}}[b] == \text{magic}$	$\forall\, b,$ chain $\boldsymbol{q}_1$	·	B	·
`lin`	$2^{36}\boldsymbol{G}_2[b] + 2^{18}\boldsymbol{g}_{1l}[b] + \boldsymbol{g}_{0l}[b] + \boldsymbol{s}_{\text{hi}}[b] == \text{magic} - 1$	$\forall\, b,$ chain $\boldsymbol{q}_2$	·	B	·
	broadcast product at S^2				
`decl`	$\boldsymbol{out}_{\text{full}}$	$\forall\, b, i$	Bd	·	·
`rescale`	$\boldsymbol{out}[b,i] \leftarrow \texttt{rescale}(\boldsymbol{out}_{\text{full}}[b,i])$	$\forall\, b, i$	$5Bd$	$2Bd$	$2Bd$
`chal`	ρ	$\forall\, i$	·	·	·
`lin`	$\boldsymbol{u}[b] \leftarrow \sum_{i\in[d]} \rho[i]\ \boldsymbol{x}[b,i]$	$\forall\, b$	B	B	·
`lin`	$\boldsymbol{p}[b] \leftarrow \sum_{i\in[d]} \rho[i]\ \boldsymbol{out}_{\text{full}}[b,i]$	$\forall\, b$	B	B	·
`quad`	$\boldsymbol{y}[b]\ \boldsymbol{u}[b] == \boldsymbol{p}[b]$	$\forall\, b$	·	·	B
totals			$7Bd + 82B$	$2Bd + 17B$	$3Bd + 42B$

The carry-chain and bracket-pin blocks are written once and run once per chain $\boldsymbol{q} \in \{\boldsymbol{q}_1, \boldsymbol{q}_2\}$; their count cells already include both chains. The per-cell broadcast products $\boldsymbol{x}[b,i]\ \boldsymbol{y}[b]$ appear in no line and are never committed: the quad pin compares projections, which is what collapses Bd Hadamard slots to B.

The totals row matches A.1's RMSNorm row. (A.1's per-row constants were updated together with this listing: the wrap-free bracket raised them from the original construction's $26B$, $7B$, $13B$ to $82B$, $17B$, $42B$, a shift far below the resolution of A.2's validation; the per-cell terms, which carry the cost, never changed.)

Soundness (Lemma B.4). The declarations are $\boldsymbol{y}$, the window words and limbs, the carry highs, $\boldsymbol{G}_2$, the slacks with their words, and $\boldsymbol{out}_{\text{full}}$. The word declarations discharge together with their ties as Lemma B.1a instances: the $\boldsymbol{y} - 1$ split bounds $\boldsymbol{y} \in [1, 2^{21}]$, so $\boldsymbol{q}_1, \boldsymbol{q}_2 \le 2^{42}$, and the limb split bounds each $\boldsymbol{S}_\ell < 2^{18}$ and $\boldsymbol{S}_{\text{tot}} < 2^{54}$, so every $\boldsymbol{H}_\ell < 2^{60} < P$. Each carry step is then itself a Lemma B.1a instance on a committed value: $\boldsymbol{H}_0$ decomposes uniquely into $(\boldsymbol{g}_{0l}, \boldsymbol{g}_{0h})$, windows $(18;\ 42)$ tiling $[0, 2^{60})$; $\boldsymbol{H}_1 + \boldsymbol{g}_{0h}$ into $(\boldsymbol{g}_{1l}, \boldsymbol{g}_{1h})$, windows $(18;\ 43)$; and the ==

pins $\boldsymbol{H}_2 + \boldsymbol{g}_{1h}$ to the 25-bit $\boldsymbol{G}_2$, whose tight window is what keeps $2^{36}\boldsymbol{G}_2$ wrap-free in the pins. The chain telescopes to $\boldsymbol{q}\ \boldsymbol{S}_{\text{tot}} = 2^{36}\boldsymbol{G}_2 + 2^{18}\boldsymbol{g}_{1l} + \boldsymbol{g}_{0l}$ exactly, so the two pins read $\boldsymbol{y}^2\boldsymbol{S}_{\text{tot}} \geq \text{magic}$ and $(\boldsymbol{y}-1)^2\boldsymbol{S}_{\text{tot}} \leq \text{magic} - 1$ as integer inequalities, not congruences. Since $\boldsymbol{q} \mapsto \boldsymbol{q}\ \boldsymbol{S}_{\text{tot}}$ is strictly increasing in $\boldsymbol{y} \geq 1$, exactly one integer $\boldsymbol{y}$ satisfies both, and the slacks are then fixed by their pins, their 59-bit window sized to the largest honest bracket step with the width condition $\text{magic} + 2^{59} < P$. Given $\boldsymbol{y}$, the projection arrows and the quad pin force $\boldsymbol{out}_{\text{full}} = \boldsymbol{x} \odot \boldsymbol{y}$ row by row except with probability $1/|F|$ per row over ρ, drawn after everything above is committed. Overflow is the rejecting case of §3.6 throughout the bracket: an $\boldsymbol{S}_{\text{tot}}$ with no limb representation below 2^{54}, or a carry value outside its window, has no satisfying assignment, and the proof rejects; the honest completeness cap is a row RMS of about 460 at the demonstrated scales.

Generalization. The elementwise gain multiply that follows the normalization in the transformer block is a separate Hadamard claim (B.8). The pre-fix bracket, which range-checked the slacks in a window wide enough to admit every field element, is unsound and documented in the negative test suite; the listing above is the repaired construction the demonstrated runs used.

B.5 SiLU

SiLU proves $\boldsymbol{out} = \boldsymbol{x}\ \sigma(\boldsymbol{x})$, saturating to $\boldsymbol{x}$ for large positive $\boldsymbol{x}$ and to 0 for large negative $\boldsymbol{x}$. The construction is a sign split, a magnitude decomposition whose low words index a table and whose high words detect saturation, a paired lookup, and a mux that swaps in the saturated value when the high words are live. The silu table is specified in the registry (bin width $\text{w}_{\text{bin}} = 4$, half-length 2^{14} per branch); the magnitude words $\boldsymbol{a}_0, \ldots, \boldsymbol{a}_4$ have widths 2, 14 (the table index), 16, 16, 14 at strides $1, \text{w}_{\text{bin}}, 2^{16}, 2^{32}, 2^{48}$.

			W	L	Q
`input`	$\boldsymbol{x}$	$\forall\, n \in [N]$	$\cdot$	$\cdot$	$\cdot$
	— sign split —				
	sign bit				
`decl`	$\boldsymbol{sign}$	$\forall\, n$	N	$\cdot$	$\cdot$
`quad`	$\boldsymbol{sign}[n]^2 == \boldsymbol{sign}[n]$	$\forall\, n$	$\cdot$	$\cdot$	N
`quad`	$\boldsymbol{C}[n] \leftarrow \boldsymbol{sign}[n] \cdot \boldsymbol{x}[n]$	$\forall\, n$	N	$\cdot$	N
`lin`	$\boldsymbol{mag}[n] \leftarrow \boldsymbol{x}[n] - 2\,\boldsymbol{C}[n]$	$\forall\, n$	N	N	$\cdot$
	— magnitude words —				
	magnitude words				
`decl`	$\boldsymbol{a}_0,\ \boldsymbol{a}_1,\ \boldsymbol{a}_2,\ \boldsymbol{a}_3,\ \boldsymbol{a}_4$	$\forall\, n$	$5N$	$\cdot$	$\cdot$
`range`	$\boldsymbol{a}_0[n] \sqsubseteq \text{range}_2, \quad \boldsymbol{a}_2[n] \sqsubseteq \text{range}_{16}, \quad \boldsymbol{a}_3[n] \sqsubseteq \text{range}_{16}, \quad \boldsymbol{a}_4[n] \sqsubseteq \text{range}_{14}$	$\forall\, n$	$4N$	$\cdot$	$4N$
`lin`	$\boldsymbol{mag}[n] == \boldsymbol{a}_0[n] + \text{w}_{\text{bin}}\,\boldsymbol{a}_1[n] + 2^{16}\boldsymbol{a}_2[n] + 2^{32}\boldsymbol{a}_3[n] + 2^{48}\boldsymbol{a}_4[n]$	$\forall\, n$	$\cdot$	N	$\cdot$
	— saturation flag —				
`lin`	$\boldsymbol{g}[n] \leftarrow 2^{16}\boldsymbol{a}_2[n] + 2^{32}\boldsymbol{a}_3[n] + 2^{48}\boldsymbol{a}_4[n]$	$\forall\, n$	N	N	$\cdot$
	free at $\boldsymbol{g} = 0$*; value-neutral*				
`decl`	$\boldsymbol{inv}$	$\forall\, n$	N	$\cdot$	$\cdot$
`quad`	$\boldsymbol{t}[n] \leftarrow \boldsymbol{g}[n] \cdot \boldsymbol{inv}[n]$	$\forall\, n$	N	$\cdot$	N
`quad`	$\boldsymbol{t}[n] \cdot \boldsymbol{g}[n] == \boldsymbol{g}[n]$	$\forall\, n$	$\cdot$	$\cdot$	N
`quad`	$\boldsymbol{t}[n]^2 == \boldsymbol{t}[n]$	$\forall\, n$	$\cdot$	$\cdot$	N
	— lookup and mux —				
`lin`	$\boldsymbol{key}[n] \leftarrow 2^{14}\,\boldsymbol{sign}[n] + \boldsymbol{a}_1[n]$	$\forall\, n$	N	N	$\cdot$
`lin`	$\boldsymbol{sat}[n] \leftarrow \boldsymbol{x}[n] - \boldsymbol{C}[n]$	$\forall\, n$	N	N	$\cdot$
`lookup`	$\boldsymbol{y}[n] \leftarrow \text{silu}[\boldsymbol{key}[n]]$	$\forall\, n$	$3N$	N	N
`quad`	$\boldsymbol{mux}_a[n] \leftarrow \boldsymbol{t}[n] \cdot \boldsymbol{y}[n]$	$\forall\, n$	N	$\cdot$	N
`quad`	$\boldsymbol{mux}_b[n] \leftarrow \boldsymbol{t}[n] \cdot \boldsymbol{sat}[n]$	$\forall\, n$	N	$\cdot$	N
`lin`	$\boldsymbol{out}[n] \leftarrow \boldsymbol{y}[n] - \boldsymbol{mux}_a[n] + \boldsymbol{mux}_b[n]$	$\forall\, n$	N	N	$\cdot$
totals			$23N$	$7N$	$12N$

The totals row matches A.1's SiLU row exactly.

Soundness (Lemma B.5). The declarations are $\boldsymbol{sign}$, the words, and $\boldsymbol{inv}$. The words are pinned by the tie with Lemma B.1a, the lookup bounding $\boldsymbol{a}_1$; the maximum recomposable magnitude is exactly $2^{62} - 1$ (widths $2, 14, 16, 16, 14$ at strides $1, 4, 2^{16}, 2^{32}, 2^{48}$). With $\boldsymbol{sign}$ boolean and $\boldsymbol{C} = \boldsymbol{sign} \cdot \boldsymbol{x}$, the two candidate $(\boldsymbol{sign}, \boldsymbol{mag})$ pairs for a given $\boldsymbol{x}$ are $(0, \boldsymbol{x})$ and $(1, P - \boldsymbol{x})$; since $\boldsymbol{x} + (P - \boldsymbol{x}) = P > 2\,(2^{62} - 1)$, at most one representative fits the bound, so the sign is unique. $\boldsymbol{inv}$: at $\boldsymbol{g} \neq 0$ the flag constraints force $\boldsymbol{t} = 1$ and $\boldsymbol{inv} = 1/\boldsymbol{g}$, unique; at $\boldsymbol{g} = 0$ they force $\boldsymbol{t} = 0$ and leave $\boldsymbol{inv}$ free, value-neutral. Everything else is an arrow: the flag, the mux, and the output follow linearly. Overflow: the decomposition is the rejecting case of §3.6; the saturated path returns $\boldsymbol{x}$ or 0 exactly.

B.6 Mixture-of-experts routing and combine

Routing pins a one-hot mask $\boldsymbol{m} \in \{0,1\}^{T \times E}$ to the argmax of committed router logits $\boldsymbol{r}$, tiebroken by expert index; the combine forms $\boldsymbol{y}[t,:] = \sum_e \boldsymbol{m}[t,e]\ \boldsymbol{X}_e[t,:]$ over the E committed expert streams without committing the $E\ T\ F$

masked products. Public constants: the tiebreak stride 2^L with $L = \lceil \log_2 E \rceil$, and the gap window $\mathrm{w_r} + L$ with $\mathrm{w_r}$ the router-logit width (demonstrated: $E = 128$, $L = 7$, $\mathrm{w_r} = 26$, three 11-bit words covering the 33-bit window).

			W	L	Q
input	$\boldsymbol{r}$	$\forall\, t \in [T],\ e \in [E]$	$\cdot$	$\cdot$	$\cdot$
input	$\boldsymbol{X}_e$	$\forall\, e,\ t,\ f \in [F]$	$\cdot$	$\cdot$	$\cdot$
	— routing —				
	the mask				
decl	$\boldsymbol{m}$	$\forall\, t, e$	TE	$\cdot$	$\cdot$
quad	$\boldsymbol{m}[t,e]^2 == \boldsymbol{m}[t,e]$	$\forall\, t, e$	$\cdot$	$\cdot$	TE
lin	$\boldsymbol{rt}[t,e] \leftarrow 2^L\, \boldsymbol{r}[t,e] + (E{-}1{-}e)$	$\forall\, t, e$	TE	TE	$\cdot$
quad	$\boldsymbol{mrt}[t,e] \leftarrow \boldsymbol{m}[t,e] \cdot \boldsymbol{rt}[t,e]$	$\forall\, t, e$	TE	$\cdot$	TE
lin	$\sum_e \boldsymbol{m}[t,e] == 1$	$\forall\, t$	$\cdot$	T	$\cdot$
lin	$\boldsymbol{r}^*[t] \leftarrow \sum_e \boldsymbol{mrt}[t,e]$	$\forall\, t$	T	T	$\cdot$
lin	$\boldsymbol{gap}[t,e] \leftarrow \boldsymbol{r}^*[t] - \boldsymbol{rt}[t,e]$	$\forall\, t, e$	TE	TE	$\cdot$
	gap words				
decl	$\boldsymbol{w}_g$	$\forall\, t, e,\ n \in [3]$	$3TE$	$\cdot$	$\cdot$
range	$\boldsymbol{w}_g[t,e,n] \sqsubseteq \mathrm{range}_{11}$	$\forall\, t, e, n$	$3TE$	$\cdot$	$3TE$
lin	$\boldsymbol{gap}[t,e] == \sum_{n\in[3]} 2^{11n}\, \boldsymbol{w}_g[t,e,n]$	$\forall\, t, e$	$\cdot$	TE	$\cdot$
lin	$\boldsymbol{r}_{\text{chosen}}[t] \leftarrow 2^{-L}\big(\boldsymbol{r}^*[t] - \sum_e (E{-}1{-}e)\, \boldsymbol{m}[t,e]\big)$	$\forall\, t$	T	T	$\cdot$
routing totals			$10TE + 2T$	$3TE + 3T$	$5TE$
chal	ρ	$\forall\, f$	$\cdot$	$\cdot$	$\cdot$
	— combine —				
	the combined output				
decl	$\boldsymbol{y}$	$\forall\, t, f$	TF	$\cdot$	$\cdot$
lin	$\boldsymbol{m}_{\text{em}}[e,t] \leftarrow \boldsymbol{m}[t,e]$	$\forall\, e, t$	ET	ET	$\cdot$
lin	$\boldsymbol{s}[e,t] \leftarrow \sum_f \boldsymbol{X}_e[t,f]\, \rho[f]$	$\forall\, e, t$	ET	ET	$\cdot$
quad	$\boldsymbol{ms}[e,t] \leftarrow \boldsymbol{m}_{\text{em}}[e,t] \cdot \boldsymbol{s}[e,t]$	$\forall\, e, t$	ET	$\cdot$	ET
lin	$\boldsymbol{ms}_{\text{tm}}[t,e] \leftarrow \boldsymbol{ms}[e,t]$	$\forall\, t, e$	ET	ET	$\cdot$
lin	$\boldsymbol{y}_\rho[t] \leftarrow \sum_f \rho[f]\, \boldsymbol{y}[t,f]$	$\forall\, t$	T	T	$\cdot$
lin	$\sum_e \boldsymbol{ms}_{\text{tm}}[t,e] == \boldsymbol{y}_\rho[t]$	$\forall\, t$	$\cdot$	T	$\cdot$
combine totals			$TF + 4ET + T$	$3ET + 2T$	ET

The two totals rows match A.1's routing and freivalds-combine rows. The combine's $4ET$ includes genuinely duplicated committed rows: expert-major and token-major copies of the mask and of the masked products, bound to each other by the exact re-indexing arrows. The duplication is a layout necessity, not slack: the pointwise quadratic and the row-sum families each need their operands in one flat layout, and the expert streams are absorbed expert-major as their matmuls retire in the streaming fold while the mask and output are token-major, so the mask crosses to expert-major for the product and the products cross back for the per-token sum.

Soundness (Lemma B.6). Routing's declarations are $\boldsymbol{m}$ and the gap words, the words discharging with their tie as a Lemma B.1a instance. The tiebreak makes all $\boldsymbol{rt}$ in a row distinct, booleanity and cardinality force $\boldsymbol{m}$ one-hot, and $\boldsymbol{r}^*$ is then the selected tiebroken logit. If the selection were not the argmax, some $\boldsymbol{gap}[t,e]$ would be a negative field element near P, which the 3×11-bit decomposition cannot recompose (width condition: $2^{33} \ll P$, given that $\boldsymbol{r}$ is bounded to $\pm 2^{\mathrm{w_r}-1}$ by its producing matmul's rescale, $\mathrm{w_r} = 26$); so $\boldsymbol{m}$ is the unique argmax mask and $\boldsymbol{r}_{\text{chosen}}$ follows linearly. The combine's declaration is $\boldsymbol{y}$: for each token, $\sum_e \boldsymbol{m}[t,e]\ \boldsymbol{s}[e,t]$ equals the projection of $\sum_e \boldsymbol{m}[t,e]\boldsymbol{X}_e[t,:]$ by the one-hotness of $\boldsymbol{m}$, so the pin forces $\boldsymbol{y}[t,:] \cdot \rho$ to match the combined stream's projection, and a false $\boldsymbol{y}$ row survives with probability $1/|F|$ over ρ, drawn after $\boldsymbol{y}$ and the streams are committed. Overflow: the gap decomposition is the rejecting case of §3.6; the projections are the accepting case.

Generalization. All E expert streams are committed even though one fires, a hiding requirement of §3.3, and the elementwise nonlinearity is applied once after the combine in the top-1 form. The routing weight applied to the chosen expert is a paired lookup against the registry's sigmoid table on $\boldsymbol{r}_{\text{chosen}}$ (B.8). A sibling of this claim, at $E = V$ but without the tiebreak and with the gap bound carried by a table lookup rather than a word decomposition, is the argmax and hidden-select machinery of the surprisal claims (B.7).

B.7 Surprisal claims

The bound of §4 is computed from the LM-head logits by a short chain of claims per output position, reusing a sibling of the routing gap machinery (B.6), the paired lookup (B.1), and the elementwise claims; Appendix E binds the same committed tokens to the digests recorded at generation time (§7.2). Its soundness property is the weaker downstream one of §7.2: the witness is deliberately not unique, and instead every prover freedom provably inflates the reported value. The tables Exp and Pow and their scales are specified in the registry (B.1.0); the ceiling divisor $\mathrm{k} = \mathrm{s_c}/\mathrm{s_b} = 2^{16}$ is a power of two, and the slack $\boldsymbol{d}$ decomposes into four 12-bit words against $d_{\max} = V\mathrm{s_y}$. All in-circuit arithmetic is in nats at scale $\mathrm{s_b}$; the public value is the revealed sum $\boldsymbol{S}_z$, and the conversion to bits, $\hat{U} = \boldsymbol{S}_z/(\mathrm{s_b} \ln 2)$ rounded up, happens outside the proof.

Per position t, with $\boldsymbol{\ell}$ the logit row and $\boldsymbol{o}$ the committed token stream:

			W	L	Q
`input`	$\boldsymbol{\ell}$	$\forall\, i \in [V]$	$\cdot$	$\cdot$	$\cdot$
`input`	$\boldsymbol{o}[t]$		$\cdot$	$\cdot$	$\cdot$
	— argmax and output select — argmax and output-select one-hots				
`decl`	$\boldsymbol{A}$, $\boldsymbol{O}$	$\forall\, i$	$2V$	$\cdot$	$\cdot$
`quad`	$\boldsymbol{A}[i]^2 == \boldsymbol{A}[i]$	$\forall\, i$	$\cdot$	$\cdot$	V
`quad`	$\boldsymbol{O}[i]^2 == \boldsymbol{O}[i]$	$\forall\, i$	$\cdot$	$\cdot$	V
`lin`	$\sum_i \boldsymbol{A}[i] == 1$		$\cdot$	1	$\cdot$
`lin`	$\sum_i \boldsymbol{O}[i] == 1$		$\cdot$	1	$\cdot$
`lin`	$\sum_i i\, \boldsymbol{O}[i] == \boldsymbol{o}[t]$		$\cdot$	1	$\cdot$
`quad`	$\boldsymbol{A\ell}[i] \leftarrow \boldsymbol{A}[i] \cdot \boldsymbol{\ell}[i]$	$\forall\, i$	V	$\cdot$	V
`lin`	$\boldsymbol{v}^* \leftarrow \sum_i \boldsymbol{A\ell}[i]$		1	1	$\cdot$
`lin`	$\boldsymbol{gap}[i] \leftarrow \boldsymbol{v}^* - \boldsymbol{\ell}[i]$	$\forall\, i$	V	V	$\cdot$
`range`	$\boldsymbol{gap}[i] \sqsubseteq \mathrm{range}_{20}$	$\forall\, i$	V	$\cdot$	V
`quad`	$\boldsymbol{Ogap}[i] \leftarrow \boldsymbol{O}[i] \cdot \boldsymbol{gap}[i]$	$\forall\, i$	V	$\cdot$	V
`lin`	$\boldsymbol{gap}_o \leftarrow \sum_i \boldsymbol{Ogap}[i]$		1	1	$\cdot$
	— kernel and log pin —				
`lookup`	$\boldsymbol{e}[i] \leftarrow \mathrm{Exp}[\boldsymbol{gap}[i]]$	$\forall\, i$	$3V$	V	V
`quad`	$\boldsymbol{g}_2 \leftarrow \boldsymbol{gap}_o \cdot \boldsymbol{gap}_o$		1	$\cdot$	1
`lin`	$\boldsymbol{a} \leftarrow \sum_i \boldsymbol{e}[i]$		1	1	$\cdot$
	the log-pin index				
`decl`	$\boldsymbol{b}$		1	$\cdot$	$\cdot$
`lookup`	$\boldsymbol{pw} \leftarrow \mathrm{Pow}[\boldsymbol{b}]$		3	1	1
	log-pin slack				
`decl`	$\boldsymbol{d}$		1	$\cdot$	$\cdot$
	slack words				
`decl`	$\boldsymbol{d}_w$	$\forall\, n \in [4]$	4	$\cdot$	$\cdot$
`range`	$\boldsymbol{d}_w[n] \sqsubseteq \mathrm{range}_{12}$	$\forall\, n$	4	$\cdot$	4
`lin`	$\boldsymbol{d} == \sum_{n\in[4]} 2^{12n}\, \boldsymbol{d}_w[n]$		$\cdot$	1	$\cdot$
`lin` $\leq$	$\boldsymbol{a} + \boldsymbol{d} == \boldsymbol{pw}$		$\cdot$	1	$\cdot$
	ceiling remainder				
`decl`	$\boldsymbol{rem}$		1	$\cdot$	$\cdot$
`range`	$\boldsymbol{rem} \sqsubseteq \mathrm{range}_{16}$		1	$\cdot$	1
`lin`	$\boldsymbol{z}_o \leftarrow \mathrm{k}^{-1}(\boldsymbol{g}_2 + \boldsymbol{rem})$		1	1	$\cdot$
`lin`	$\boldsymbol{surprisal}[t] \leftarrow \boldsymbol{z}_o + \boldsymbol{b}$		1	1	$\cdot$
	— across positions —				
`lin`	$\boldsymbol{S}_z \leftarrow \sum_{t\in\text{scored}} \boldsymbol{surprisal}[t]$		1	1	$\cdot$
`lin`	$\boldsymbol{S}_z ==$ the revealed public value		$\cdot$	1	$\cdot$
totals per position			$9V + 22$	$2V + 13$	$6V + 7$

The output tokens enter only as the committed stream $\boldsymbol{o}$ consumed by the select pins; they never appear in the public claim list, and the indicator rows $\boldsymbol{O}$ are shared with the input selection (Appendix E.4), so the scored tokens are the tokens the model consumed. The cross-position sum is realized as chained adds over the scored positions, and $\boldsymbol{S}_z$ stays private until the final pin reveals it.

The totals row sums the listing per position; the V-length families dominate at $9V$ slots. The implementation additionally carries a negated copy of the gap family (V slots and V linears) and realizes the cross-position sum as chained adds over the scored positions. All of this machinery is excluded from the cost model by construction (A.6); it is about 0.1% of the per-token witness.

Soundness (Lemma B.7, one-sided). The declarations are $\boldsymbol{A}$, $\boldsymbol{O}$, $\boldsymbol{b}$, $\boldsymbol{d}$ with its words, and $\boldsymbol{rem}$. $\boldsymbol{O}$ is pinned uniquely by booleanity, cardinality, and the index binding: a one-hot vector with $\sum_i i\, \boldsymbol{O}[i] = \boldsymbol{o}[t]$ is exactly the indicator of $\boldsymbol{o}[t]$. $\boldsymbol{A}$ with the gap non-negativity forces $\boldsymbol{v}^* = \max_i \boldsymbol{\ell}[i]$: any non-maximal selection makes some $\boldsymbol{gap}[i]$ negative, which the gap range and the Exp lookup's key range both exclude. $\boldsymbol{A}$ itself is not unique when maximal logits tie, and no tiebreak is imposed; the freedom is value-neutral, since every valid selection yields the same $\boldsymbol{v}^*$, hence the same gaps, the same $\boldsymbol{e}$, and the same reported value, so it is a permitted downstream freedom under §7.2's one-sided rule (it neither inflates nor deflates). The remaining freedom is $\boldsymbol{b}$, and every free direction inflates. Each table entry $\boldsymbol{e}[i]$ rounds the true exponential up and is floored at one, so $\boldsymbol{a}$ over-counts the true normalizer; Pow rounds down, so the one-sided pin $\boldsymbol{a} \leq \mathrm{Pow}[\boldsymbol{b}]$ forces $\boldsymbol{b} \geq \mathrm{s_b} \ln(\boldsymbol{a}/\mathrm{s_y})$ with no fractional escape, and choosing $\boldsymbol{b}$ above the least valid index only raises the reported value; $\boldsymbol{z}_o$ is a ceiling, its slack $\boldsymbol{d}$ fixed by the pin once $\boldsymbol{a}$ and $\boldsymbol{pw}$ are and non-negative by its decomposition. With $Q(o) = \boldsymbol{e}_o/\boldsymbol{a}$, a genuine distribution since $\boldsymbol{a}$ normalizes the committed table values exactly, $\boldsymbol{z}_o + \boldsymbol{b} \geq \mathrm{s_b}(-\ln Q(o))$ follows term by term, so $\boldsymbol{S}_z$ upper-bounds the true surprisal sum and a poor witness penalizes only the prover. Two freedoms need field arguments rather than integer ones. The word decomposition of $\boldsymbol{d}$ is safe by width: at four 12-bit words against $d_{\max} = V\mathrm{s_y}$ the maximum recomposable value lies far below the modulus, so no wrapped negative $\boldsymbol{d}$ has a valid decomposition (Lemma B.1a). The ceiling arrow is subtler: over the integers $\boldsymbol{z}_o$ is unique given $\boldsymbol{g}_2$, but over the field every range-valid remainder admits a solution $\boldsymbol{z}_o = \mathrm{k}^{-1}(\boldsymbol{g}_2 + \boldsymbol{rem}) \bmod P$,

and $\boldsymbol{z}_o$ carries no range check. The reachable perturbations of the public sum are $\boldsymbol{S}'_z = \boldsymbol{S}_z + \mathrm{k}^{-1}s \bmod P$ for integer $s \in [0, T\mathrm{k})$; deflating by any amount requires $s = P - \mathrm{k}\Delta$, on the order of 2^{64} and unreachable by roughly twenty orders of magnitude, while every reachable perturbation either inflates the bound by less than T scaled-nat units, i.e. $T/\mathrm{s_b}$ nats (at $\mathrm{s_b} = 2^{12}$ over the demonstrated 500 scored positions, about 0.12 nats or 0.18 bits, the safe direction) or lands $\boldsymbol{S}_z$ near the modulus, an absurd self-reported bound useless to a deflating prover. The claim is sound by exclusion rather than by uniqueness; this is the one place in the construction where that argument is load-bearing.

Generalization. Normalization is where the asymmetry between §7.2's two rules pays: softmax pins its shift exactly because upstream slack is unanalyzable, while the bound replaces normalization with the one-sided logarithm pin because downstream slack provably only inflates. Summing over a subset of positions bounds the unexplained information of just those outputs; the demonstrated runs score the 500-token continuation.

B.8 Remaining claims

The remaining claim types are compositions of B.1 machinery with no soundness argument beyond Lemma B.1a and the arrow rule; their listings are omitted and their counts, matching A.1, are:

claim	built from	W	L	Q
add (N)	nothing shared	N	N	0
hadamard (N) ⓡ	rescale	$6N$	$2N$	$3N$
rope (N) ⓡ	rescale; public cos/sin coefficients	$6N$	$3N$	$2N$
paired lookup (N)	paired lookup	$3N$	N	N
word extraction (N, t)	word decomposition	$2tN$	N	tN
embedding select (T, V)	routing claim at $E = V$ (B.6) for one-hot validity; scale-free Freivalds matmul against the embedding matrix	$O(TV)$	$O(TV)$	$O(TV)$
masked combine (T, E, F)	committed products form	$2ETF + TF$	$ETF + TF$	ETF

The sigmoid routing weight of the MoE layers is a paired-lookup instance against the registry's sigmoid table, one query per token per MoE layer. The masked combine is the committed-products alternative the demonstrated runs replace with B.6's projected form. The token-binding circuits (AES, SHA-256) are specified in Appendix E at their own granularity.

Appendix C. Constraint compilation and evaluation

This appendix specifies how the flat constraint system of §3 is represented and evaluated: the generative constraint level shared by the prover and the verifier, the run structure both evaluation loops exploit, the challenge-access discipline, and the two folds themselves. The governing constraint is scale. The linear system has $\Theta(\mathrm{nnz})$ nonzeros with $\mathrm{nnz} \approx 2\text{–}4\,W$, tens of terabytes at the scale of §9, so no materialized form of the constraints ever exists on either side. Everything below is regenerated on demand from descriptors whose total size is $O(n_{\text{claims}})$, a few tens of megabytes at 400B scale.

C.1 Constraint level: bands and quadratic descriptors

Each claim compiles, independently on each side (the verifier recompiles from the public claim list and never reads prover-supplied constraints, §6.5), into three kinds of object:

- **Linear bands.** Each variable carries a small list of bands, one per constraint pattern it participates in: a kind tag, a constraint-id base, and the kind's parameters (roughly 100 bytes). A band maps each flat slot f of its variable to constraint ids and coefficients by closed-form index arithmetic; the map is a pure function of the descriptor and the variable's geometry, so any row window can be evaluated without state. Parameter vectors (Freivalds ρ, λ; lookup-table coefficients; RoPE tables) are sized $O(k)$ or $O(\text{table})$, never $O(\text{nnz})$, and are shared, not copied per row.

- **Quadratic descriptors.** One per quadratic emission: the three operands' starting rows, the uniform (a, b) coefficients, the slot count L, and a positional index base. Row t of a descriptor is the per-row constraint $w_{x+t} \circ w_{y+t} + a\, w_{z+t} = b$ over $\min(\mu,\ L - t \cdot \mu)$ slots, and its combiner index is base $+ t$. This replaces a per-row constraint list of size $O(W/\mu)$, a verifier-memory binder at long context, with $O(n_{\text{emissions}})$.
- **Right-hand sides**, kept as compact runs $(\text{start}, \text{length}, \text{value})$.

Constraint ids and quadratic indices advance in claim order; this positional numbering is the entire cross-side contract. The fold combiners $r_{\text{lin}}[g]$ and $r_{\text{quad}}[t]$ are values of a hash PRF on $(s_{\text{comb}}, \text{index}, \text{label})$ and are never materialized globally: both sides derive any combiner in $O(1)$ from the round seed, so no challenge vectors cross the wire. Because a quadratic descriptor carries its index base, firing order is immaterial: each row fetches its own combiner, and field addition commutes.

C.2 Run structure and challenge access

A band's slot-to-(id, coefficient) map decomposes into maximal homogeneous runs of four shapes, and the shape is a static property of the band kind:

shape	structure	challenge access
repeat	a run of slots shares one id	one PRF call per run
strided repeat	id $=$ base $+ (f \bmod k)$: k distinct ids recur on every row	preload $[\text{base}, \text{base} + k)$, cached
one-to-one	id $=$ base $+ f$ (RoPE: stride 2)	one PRF call per slot
fan	one slot feeds a contiguous id range	streamed range sum

Row sums and the Freivalds B/C sides are repeats; the Freivalds A side is the strided repeat, whose ids have no contiguous runs but span only $k \le H \cdot K$ ids (≤ 128 KB preloaded; the prover caches these buffers for the whole proof, across all chunks and layers, since s_{comb} is fixed once per round). Identity pins and lookups are one-to-one, where one hash per distinct id is the floor; broadcasts are fans, where only the range *sum* is needed, so the range is never buffered. The effect is that challenge hashing costs $O(\text{distinct ids})$ on the duplication-heavy bands rather than $O(\text{nnz})$: a weight matmul's B-side reuses each id n times, an expert matmul's A-side m times.

C.3 Prover fold (round 3)

The linear test polynomial is $q_{\text{lin}} = \sum_i R_i \cdot p_i$, where p_i is row i's committed codeword polynomial and R_i interpolates row i's slice of $r_{\text{lin}}^{\top} A$. The prover computes it during the same tape-order sweep that regenerates the witness (§6.3), in two stages per 256-row chunk:

1. **Band evaluation** (kind-specific, per band, internally uniform): the band index (descriptors sorted by their variables' disjoint row ranges) yields the bands overlapping the chunk in one binary search; each band evaluates its intersection window into the chunk's $r^{\top} A$ rows, reading challenges per its shape.
2. **The transform fold** (kind- and variable-agnostic, batched): interpolate the chunk's $r^{\top} A$ rows to coefficients (inverse NTT), forward-transform both factors, multiply pointwise, and accumulate the products in the evaluation domain; one inverse NTT at the end of the fold recovers q_{lin} (exact, since the inverse transform is linear), replacing a per-row inverse transform. Rows are freed afterwards, preserving the working-set memory bound.

Quadratic descriptors fire at their declaring claim, where the sweep's value liveness guarantees all three operands are resident: their rows are re-encoded on demand (exact, because the zero-knowledge padding of §6.1 is generated by a PRG seeded with the absolute row index, so re-encoding reproduces the committed polynomial bit for bit), and the pointwise products fold into p_0 under the positionally indexed combiners.

C.4 Verifier evaluation

The verifier recompiles the same bands and quadratic descriptors from the public claim list, then evaluates them at the opened columns rather than over full polynomials. The linear sum check needs no witness at all: $\sum_c q_{\text{lin}}(\zeta_c)$ is compared against the right-hand-side runs, each run one PRF range sum. The linear column check reconstructs, for every opened point η_j, each row's $R_i(\eta_j)$ through the closed form for a message slot's contribution to a codeword value

(no NTT), as one generic fold over the runs: a repeat run takes one challenge and a prefix-summed-Lagrange difference (constant coefficients) or a coefficient dot (vector coefficients, with the Freivalds λ factored out per run); strided repeats read the band's preload; a fan takes one challenge range sum shared across all τ opened points. The quadratic column check walks the quadratic descriptors' rows with their positional combiners.

One implementation of the index arithmetic sits on the verdict path. A second, per-term generator, line-for-line with the prover's, exists only as a test oracle: property tests compare the two as complete (slot, id, coefficient) triple sets over adversarial row windows for every band kind, so the trusted base carries a single copy of the geometry while its correctness is checked against an independent one.

C.5 Equivalence discipline

Every representation change above (per-row structures to bands and descriptors, per-term evaluation to runs, inline hashing to preloads) is a regrouping of exact field operations, so equality of results is exact, not approximate, and the development gates demand it: verifier builds are compared by per-check *values* (not verdicts) on stored proofs; the prover's fold was compared chunk-by-chunk against an unmodified reference path until its retirement; tampered proofs must still be rejected after every change; and cross-language agreement is tested by expanding both compilers' outputs to canonical triples. The positional numbering of C.1 is what makes this possible: no reorganization changes which challenge multiplies which constraint, so any drift in any bit is a defect by definition.

Appendix E. Token binding

The bound of §4 is conditioned on the input tokens and scored on the output tokens, so it certifies the real run only if the committed streams are the streams the run actually used (§7.2). Inside the proof they are ordinary hidden witness: the claims force the output tokens to be *some* valid selections, not the ones the deployment emitted, and a prover could commit a lower-surprisal transcript and deflate the bound, or condition on a fabricated prompt and certify nothing. This appendix gives the construction that closes the gap by binding both committed streams to a commitment recorded independently at generation time, outside the proof.

E.1 Recorded commitment

At generation time, an independent recording process computes, for each request/response exchange,

$$H_{1,\text{in}} = H(\text{AES}(\text{key}, \boldsymbol{x})), \qquad H_{1,\text{out}} = H(\text{AES}(\text{key}, \boldsymbol{o})), \qquad H_2 = H(\text{key material}),$$

with H = SHA-256 and AES in counter mode. These three digests are public inputs to the proof. One key covers the exchange, and H_2 is fixed with the request, before the response exists, so the key material cannot be chosen after the fact to fit a covert payload. The prover commits the tokens and the key material in the first round, hidden behind the Merkle root like the weights, and proves

$$H(\text{AES}(\text{key}, \boldsymbol{x})) = H_{1,\text{in}}, \qquad H(\text{AES}(\text{key}, \boldsymbol{o})) = H_{1,\text{out}}, \qquad H(\text{key material}) = H_2.$$

E.2 Soundness and the root of trust

Both digests are required. H_1 alone is vacuous: it fixes the ciphertext C, but for any key the prover grinds, decrypting C under it yields *some* token stream that re-encrypts to C, so the tokens would be free. H_2 pins the key by collision resistance; with the ciphertext and the key both fixed, the token stream is the unique decryption. The binding therefore has the stronger of §7.2's two properties, a unique satisfying assignment, obtained from collision resistance rather than from constraint structure.

What the binding then certifies is exactly this: *the committed tokens equal the ones that produced the pre-recorded digests*. That is meaningful only if the record is produced by a process the verifier trusts, independently of and prior to the proof: for example, network-boundary hardware that hashes traffic as it passes and certifies the digests on a fixed schedule. A record the prover can rewrite after the fact makes the binding circular, and the assumption should be stated wherever the bound is reported.

E.3 Confidentiality

Token streams are low-entropy, about 18 bits per token against a 202,048-token vocabulary, so a bare hash of the tokens would be a dictionary-searchable commitment and would leak them. Encrypting under a high-entropy committed key first makes H_1 a hiding commitment, preserving zero-knowledge: the digests reveal nothing about the tokens beyond their length. A deployment that chooses to reveal one stream simply publishes it alongside the proof and drops the hiding for that side; the binding equations are unchanged.

E.4 One committed integer per token

The committed token integer t_i is the single interface between the model side and the wire side of the proof, and every connection is a copy constraint on shared witness slots. On the input side, a select claim commits an indicator row M_i over the vocabulary with booleanity $M_i \circ M_i = M_i$, cardinality $\sum_j M_{ij} = 1$, and the index binding $t_i = \sum_j j \cdot M_{ij}$ (the three together pin M_i uniquely as the indicator of t_i), and the embedded stream is $x = ME$ by a Freivalds matmul against the embedding matrix, which is a committed model weight in any case. On the output side, the argmax claim's select gadget (B.7) carries the same index binding, so the observed-token gap that feeds the surprisal is evaluated at t_i by construction. On the wire side, a word decomposition splits each t_i into a fixed serialization of four little-endian bytes, which feed the cipher. Each of these links is load-bearing: without any one of them, the binding would attest to a different token set than the one the bound is computed over.

E.5 Cipher and hash in constraints

Both primitives are lookup arguments over the existing table machinery, and all values stay below 2^{32}, so the width argument of B.1 rules out field wraparound throughout. AES-128-CTR costs, per 16-byte block: sixteen S-box paired lookups per round against a 256-entry table; MixColumns as an *xtime* lookup plus linear constraints (doubling in $\mathrm{GF}(2^8)$); AddRoundKey and all other XORs against a 2^{16}-entry byte-pair table; ShiftRows as wiring. The key schedule reuses the S-box table and is proven once per key. SHA-256 costs, per 64-byte block: the σ/Σ/Ch/Maj functions on 16-bit-limb XOR and AND tables, rotations as decomposition rewiring, and mod-2^{32} additions with one range-checked carry word each; message padding is public structure compiled into the constraints.

Counter mode is chosen for compatibility in both directions. Hardware that encrypts with AES-GCM needs no additional circuit support: GCM's ciphertext is exactly counter-mode output, with counter blocks derived from the IV, which rides in the committed key material behind H_2, so no $\mathrm{GF}(2^{128})$ authentication arithmetic enters the proof. The 16-byte GCM tag cannot be partially explained inside a hash preimage, so either the recorded payload is defined as ciphertext-only, or the tag enters the preimage as unconstrained witness and is charged to the reported bound at 128 bits per packet. The fixed four-byte serialization keeps every token position-addressable and inside a single keystream block, so the same byte layout serves the circuit, the recorder, and any external process that re-derives per-token ciphertext units from the record.

E.6 Cost

The binding runs over kilobytes of tokens, not the model. A few-thousand-token transcript costs on the order of 10^6 constraint rows for both streams together, under 0.01% of the forward proof's witness (Appendix A) and invisible in the cost model's terms. SHA-256 and AES are deliberately standard rather than arithmetization-friendly choices: at token scale their circuit cost is negligible, and standard primitives are what independent recording hardware produces.